

Data Structures for Digital Information Retrieval Systems

Academic Year 2025/2026

Luigi Santangelo
luigi.santangelo@unipv.it

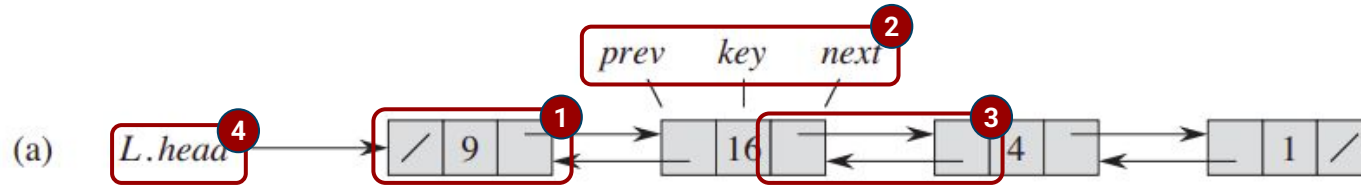
University of Pavia
**Master Degree in
Computer Engineering**

Data Structures

- Linked List
- Hash Table
- Binary Tree
- B-Tree
- Binary Heap

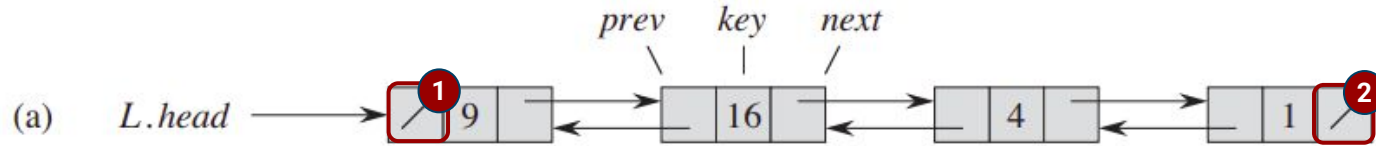
Linked List

Linked List



1. Data structure for managing objects in a **linear order**
2. Each item contains the **key** (can be anything, integer, double, string, complex data) and **two pointers** to move back and forth
3. Given an element *x* in the list, we can move to its **successor and predecessor**
4. A pointer points to the **head of the list**

Linked List

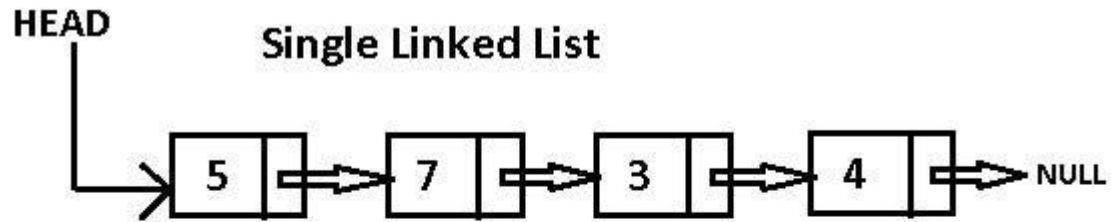


1. The predecessor of the first item is NULL
2. The successor of the last item is NULL

Linked List

A list may have several forms:

- **Singly Linked List:** no predecessor is stored in each item



Linked List

A list may have several forms:

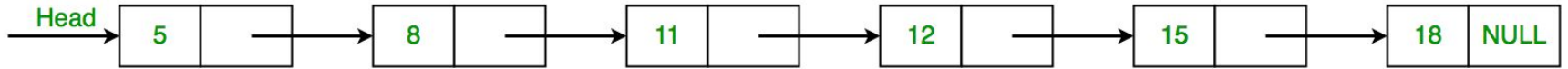
- **Circular Linked List:** the successor of the last item is linked to the the first item and the predecessor of the first item is linked to the last item



Linked List

A list may have several forms:

- **Sorted Linked List:** all the elements are sorted in non decreasing order



Sorted Linked List: searching operation

```
search(L, k)
{
    x = L.head;
    while (x != NULL && x.key < k)
        x = x.next;
    if (x != NULL && x.key == k) return x;
    return NULL;
}
```

L: the list

k: the key to look for

Complexity: **$O(n)$**

Sorted Linked List: inserting operation

```
insert(L, x)
{
    tmp = L.head
    // insert at the first position
    if (tmp == null || tmp.key >= x.key) {
        x.next = tmp
        L.head = x
    } else {
        while (tmp.next != NULL && tmp.next.key < x.key)
            tmp = tmp.next
        x.next = tmp.next
        tmp.next = x
    }
}
```

L: the list

x: the item to be inserted

Complexity: **$O(n)$** (worst case)

Sorted Linked List: deleting operation

```
delete(L, k)
{
    if (L.head == null) return;
    if (L.head.key == k)
    {
        L.head = L.head.next;
        return;
    }
    c = L.head;
```

```
        while (c.next != null &&
               c.next.key <= k)
        {
            if (c.next.key == k)
            {
                c.next = c.next.next;
                return;
            }
            c = c.next;
        }
        return;
    }
```

L: the list

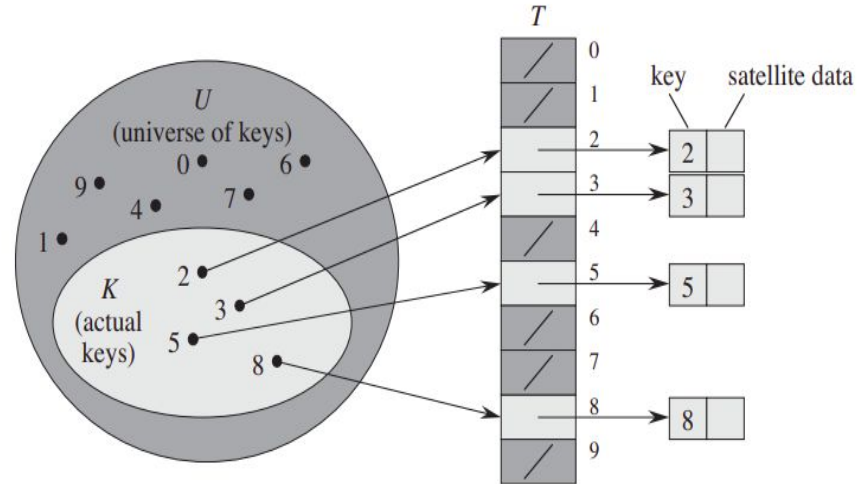
k: key to be deleted

Complexity: **O(n)** (worst case)

Hash Table

Hash Table

- Data structure for retrieving data in $O(1)$ time
- A key is used to access the array directly
- An Hash Table can be easily implemented using an Array



Hash Table: operations

```
insert(T, x)
{
    T[x.key] = x;
}
```

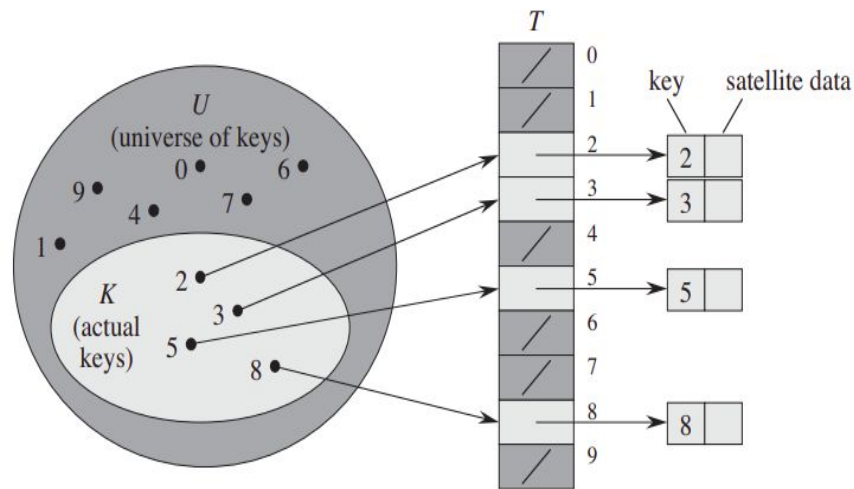
```
search(T, k)
{
    return T[k];
}
```

```
delete(T, x)
{
    T[x.key] = null;
}
```

Complexity: $O(1)$

Hash Functions

- If we use the key for accessing the Hash Table, **the size of the Hash Table** is $|U|$ where U is the universe of keys
- If $|U|$ is too large, we cannot store such **amount of data** in our memory
- The idea is to use an **hash function h** , which maps each key in the Universe with another key belonging to the set H where $|H| \ll |U|$



Hash Table: operations with hash function

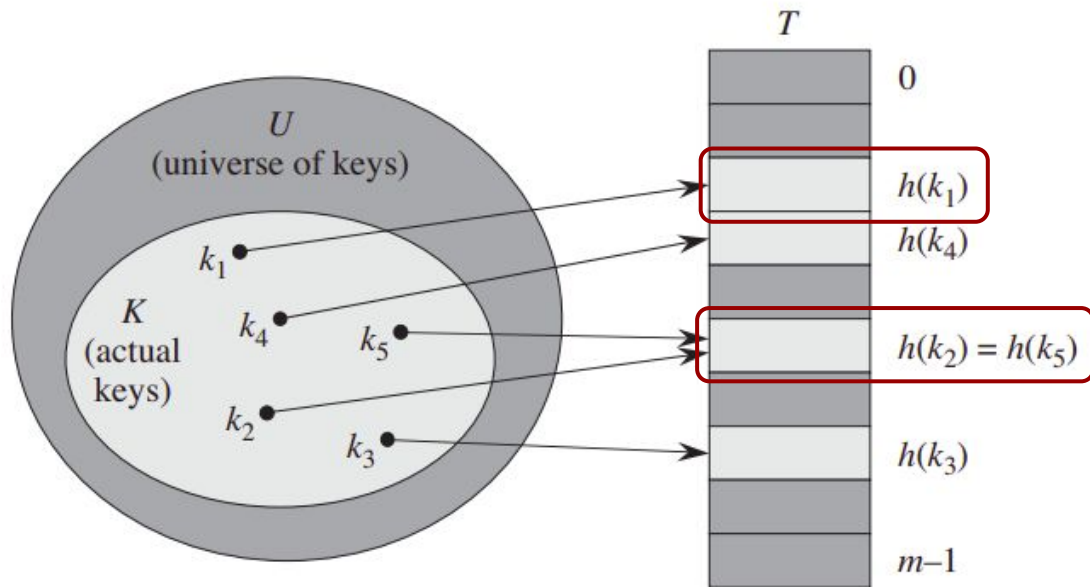
```
insert(T, x)
{
    T[h(x.key)] = x;
}
```

Complexity: $O(1)$

```
search(T, k)
{
    return T[h(k)];
}
```

```
delete(T, k)
{
    T[h(k)] = null;
}
```

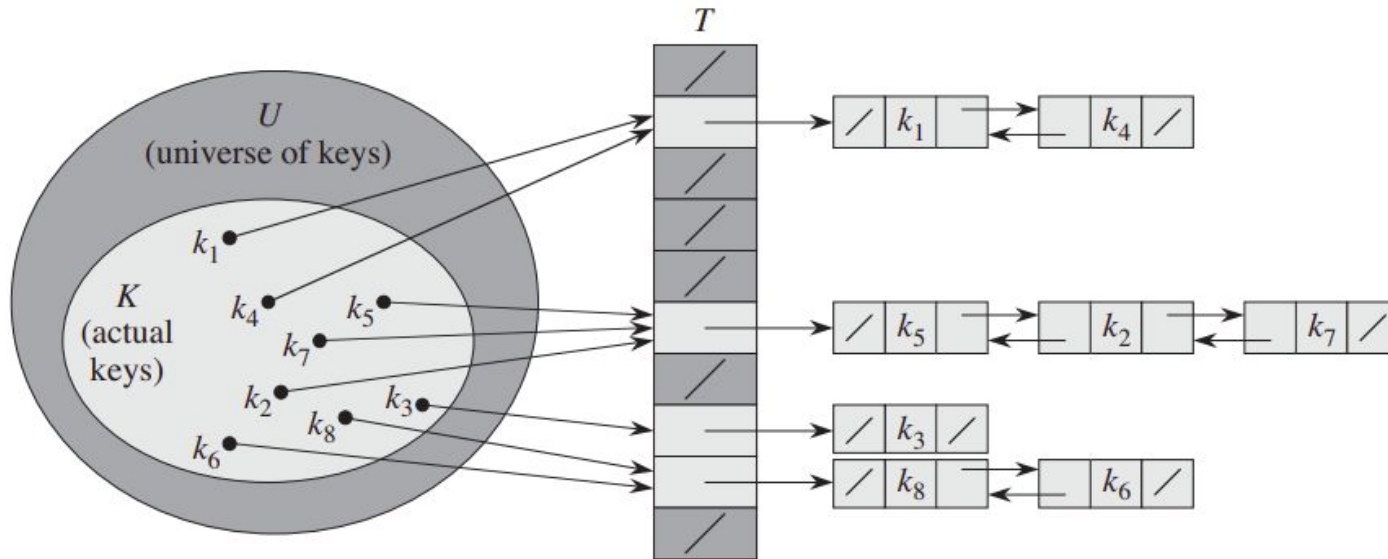

Collision



How to avoid collision

1. Be aware: **avoiding collision** is impossible!!!!
2. Using suitable hash functions to **minimizing collisions**
3. **Chaining**: place all the elements that hash to the same slot into the same linked list (see next picture)

Chaining



Hash Table: operations with chaining

```
insert(T, x)
{
    insert x at the head of the list T[h(x.key)];
}
```

Complexity: $O(1)$

```
search(T, k)
{
    search for an element with key k in list T[h(k)];
}
```

Complexity: it depends on the list length (see later)

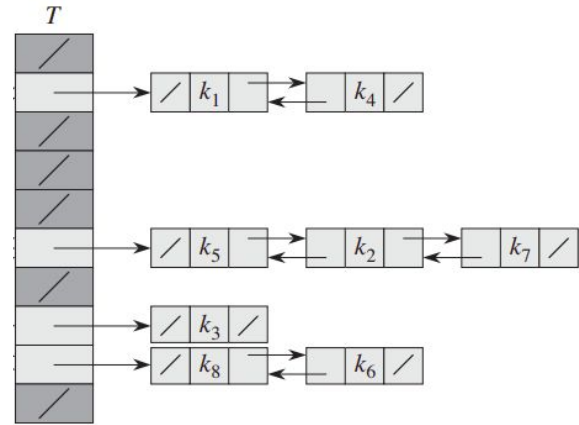
```
delete(T, k)
{
    delete x from the list T[h(k)];
}
```

Complexity: it depends on the list length (see later)

Complexity of the searching operation

- Let T be the Hash Table
- Let m the number of slots available in T
- Let n the number of elements currently stored in T
- Let $\alpha = n/m$ the **load factor** (representing the average number of element stored in a chain)

$$m = 10, n = 8, \alpha = 8/10 = 0.8$$



Complexity of the searching operation

- Worst case: all n keys end up to the same slot. The time for the searching operation is $O(n)$
- Average case: it depends on how well the hash function h is **able to distribute the set of keys** to be stored among the m slots.
- It is well known that the total time required for a successful search is

$$O(1 + \alpha)$$

Hash function: the division method

Let m be the size of the Hash Table, an hash function can be

$$h(k) = k \bmod m$$

Usually, m should be a **prime value** (to distribute evenly all the keys).

For example, we have an universe of 2000 elements, and we agree to have 3 elements on each chain. So, we need to use a prime value not too close to $2000/3$.

Then, we use $m = 701$

$$h(k) = k \bmod 701$$

Hash function: the multiplication method

- Let A be a constant in the range $0 < A < 1$
- Multiply k by A
- Take the decimal part of kA
- Multiply kA by m
- Take the floor

$$h(k) = \text{floor}(m(kA \bmod 1))$$

Hash function: the multiplication method

The advantage of this method is that **m is not critical**.

Usually we say **$m = 2^p$** , for some p .

For example $A = (\text{sqrt}(5) - 1) / 2 = 0.618033989$

$p = 14, m = 2^{14} = 16384$

$h(123456) = \text{floor} (16384 * (123456 * 0.618033989 \bmod 1)) =$

$\text{floor} (16384 * 0,004145984) = 67$

A little remind about Recursion

What is a recursive function?

```
private static int factRecursive(int num){
```

```
    if(num < 0){  
        return -1;  
    }
```

```
    if(num == 1 || num == 0){  
        return 1;  
    }
```

```
    num = num * factRecursive(num-1);  
    return num;  
}
```

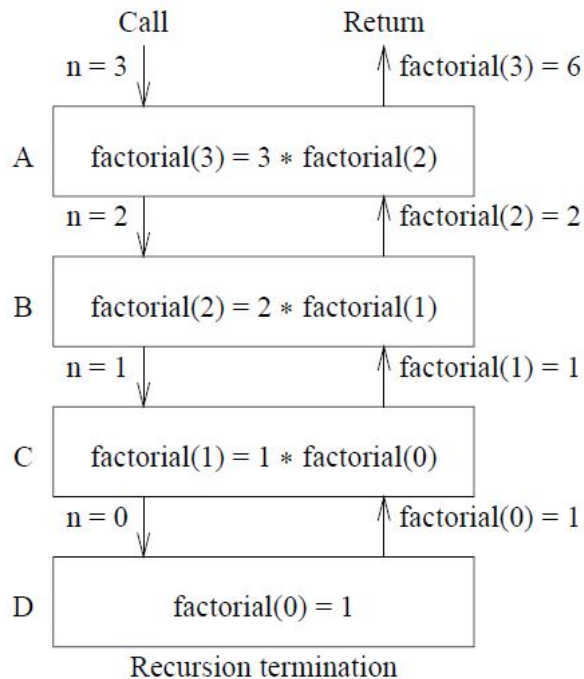
Recursive Method

Base Case (necessary condition to break recursive call)

Recursive call to same method

Recursive Function: Factorial

```
int factorial(int num)
{
    # base case
    if (num == 1)
        return 1;
    # recursive step
    else
        return num * factorial(n-1);
}
```



(a)

Recursive Function: Fibonacci sequence

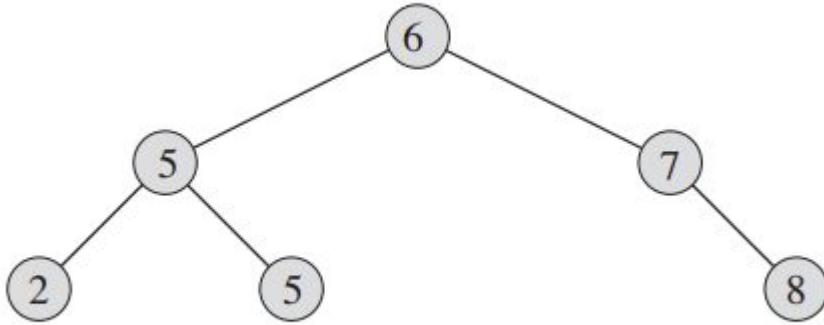
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, ...

```
int fibonacci(int n) {  
    if(n == 0) {          # base case 1  
        return 0;  
    } else if(n == 1) {   # base case 2  
        return 1;  
    } else {              # recursive step  
        return (fibonacci(n-1) + fibonacci(n-2));  
    }  
}
```

Binary Tree

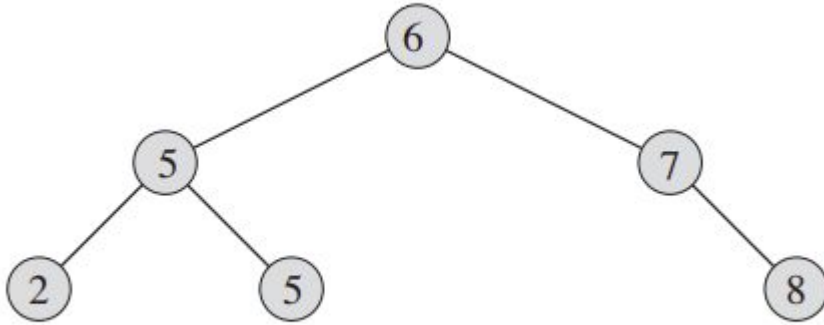


Binary Tree: properties



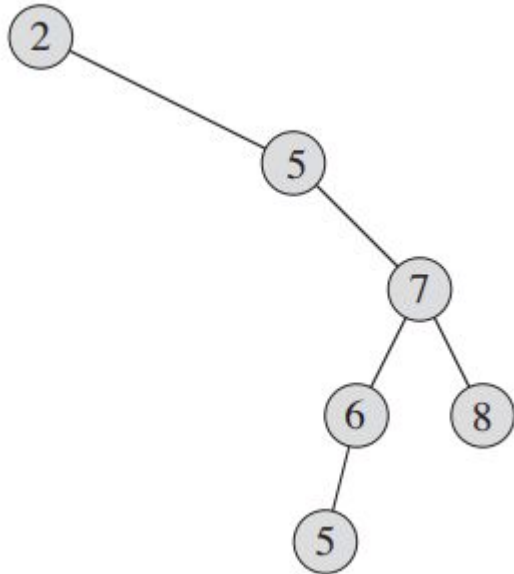
1. Each node can have 0, 1 or 2 children
2. All the children on the left subtree have a key $\leq$ than the key of the root
3. All the children on the right subtree have a key $\geq$ than the key of the root

Binary Tree: properties



- If the tree is balanced, the worst time for searching a key is $O(\log n)$

Unbalanced Binary Tree



- If the tree is completely unbalanced, the worst time for searching a key is $O(n)$, it works like a list

Inorder Tree Walk

```
inorderTreeWalk(x)
{
    if (x != NULL)
    {
        inorderTreeWalk(x.left);
        print x.key;
        inorderTreeWalk(x.right);
    }
}
```

- Recursive procedure
- x is the root of the Binary Tree

Complexity: $O(n)$

Search

```
treeSearch(x, k)
{
    if (x == NULL || k == x.key)
        return x;
    if (k < x.key)
        return treeSearch(x.left, k)
    else
        return treeSearch(x.right, k)
}
```

- Recursive procedure
- x is the root of the Binary Tree
- k is the key we are looking for

Complexity: $O(\log n)$

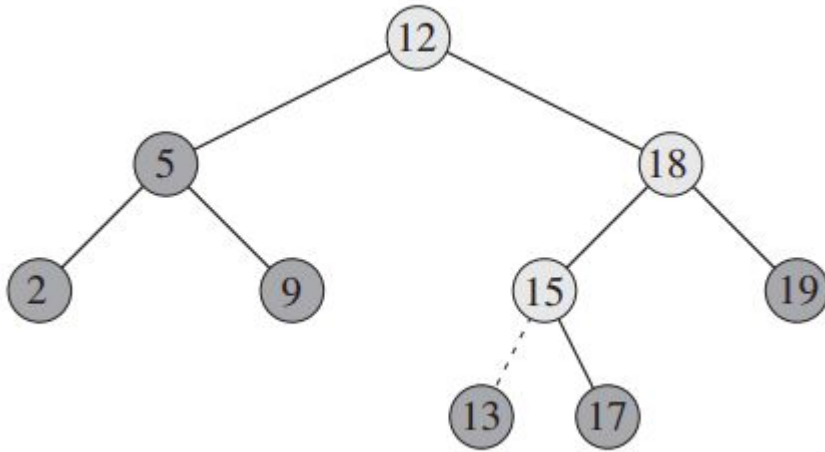
Search

```
treeSearch(x, k)
{
    while (x != null && x.key != k)
    {
        if (k < x.key)
            x = x.left;
        else
            x = x.right;
    }
    return x;
}
```

- Iterative procedure
- x is the root of the Binary Tree
- k is the key we are looking for

Complexity: $O(\log n)$

Insert



- We want to insert 13
- We walk down the tree, till reaching 15, whose left child is null
- We add 13 as left child

Insert

```
treeInsert(T, z)
{
    x = T.root
    y = NULL

    while (x != NULL)
    {
        y = x
        if (z.key < x.key)
            x = x.left
        else
            x = x.right
    }
    z.parent = y
```

```
    if (y == null)
        T.root = z
    else if (z.key < y.key)
        y.left = z;
    else
        y.right = z
```

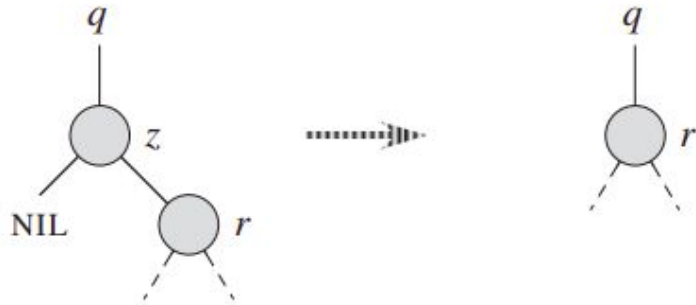
Complexity: $O(\log n)$

Deletion

Deleting a node in a binary tree is not easy as Inserting

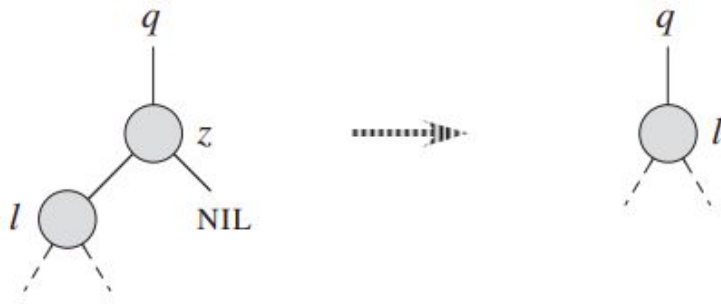
Four different cases

Deletion: case 1



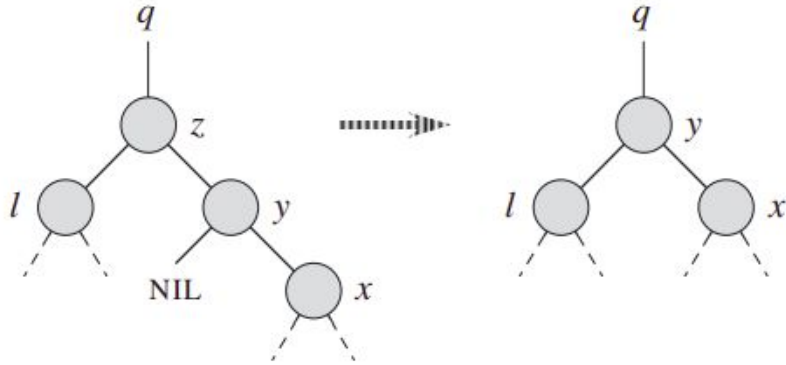
- We want to delete z
- If z has not left child, we replace z with the right child (this works even when z has no right child)

Deletion: case 2



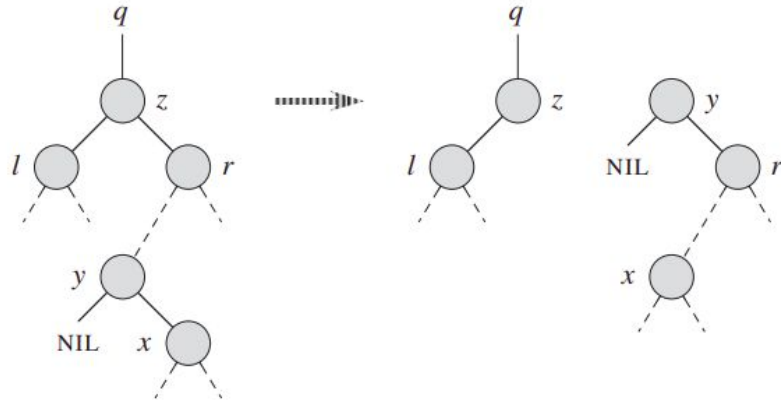
- We want to delete z
- If z has not right child, we replace z with the left child (this works even when z has no right child)

Deletion: case 3



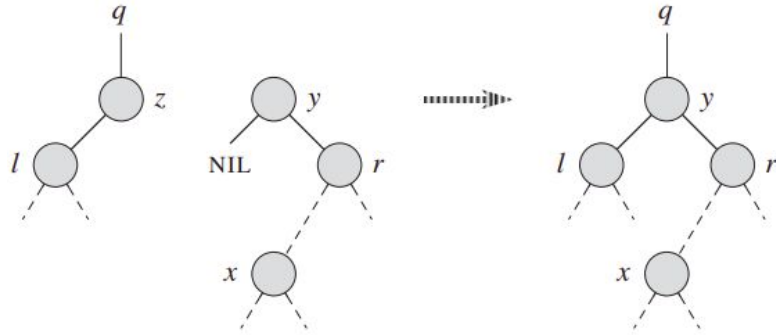
- We want to delete z
- Node z has both left and right child.
- We find z 's successor y which lies in z 's right subtree and has no left child
- if y is the right child of z , we replace z with y , putting the z 's left subtree as the y 's left subtree

Deletion: case 4 step a



- We want to delete z
- Node z has both left and right child.
- We find z 's successor y which lies in z 's right subtree and has no left child, but y is not the z 's right child
- we cut the z 's right child, we put x as left child of r , and r as the right child of y

Deletion: case 4 step b



- we put z 's left child as y 's left child
- we replace z with y

Deletion

TREE-DELETE(T, z)

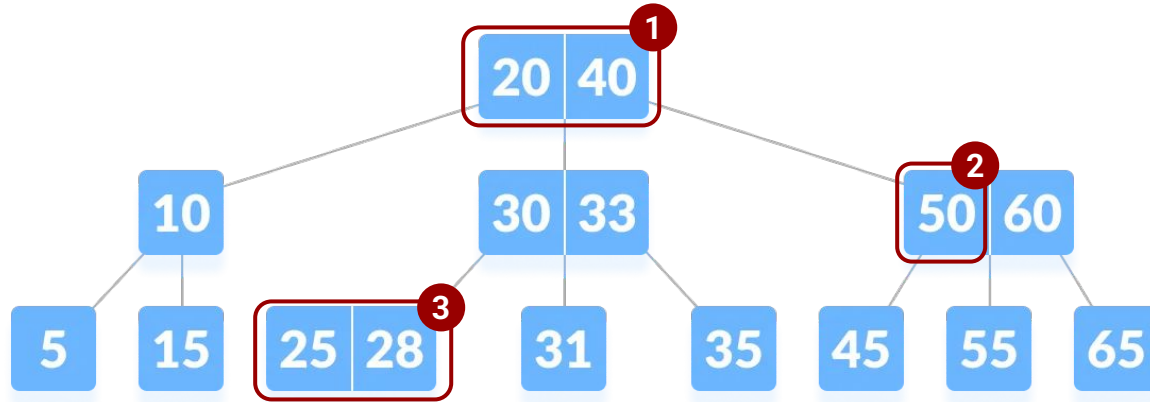
```
1  if  $z.left == \text{NIL}$ 
2      TRANSPLANT( $T, z, z.right$ )
3  elseif  $z.right == \text{NIL}$ 
4      TRANSPLANT( $T, z, z.left$ )
5  else  $y = \text{TREE-MINIMUM}(z.right)$ 
6      if  $y.p \neq z$ 
7          TRANSPLANT( $T, y, y.right$ )
8           $y.right = z.right$ 
9           $y.right.p = y$ 
10     TRANSPLANT( $T, z, y$ )
11      $y.left = z.left$ 
12      $y.left.p = y$ 
```

TRANSPLANT(T, u, v)

```
1  if  $u.p == \text{NIL}$ 
2       $T.root = v$ 
3  elseif  $u == u.p.left$ 
4       $u.p.left = v$ 
5  else  $u.p.right = v$ 
6  if  $v \neq \text{NIL}$ 
7       $v.p = u.p$ 
```

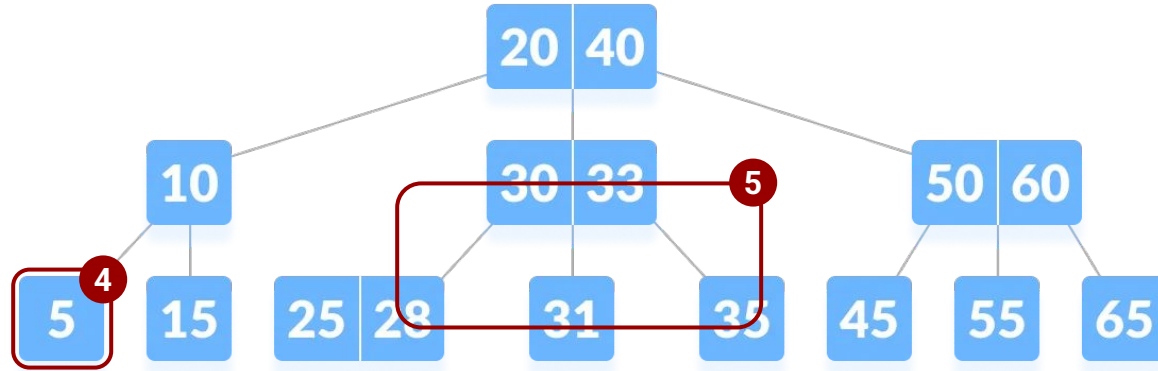
B-Tree

B-Tree: properties



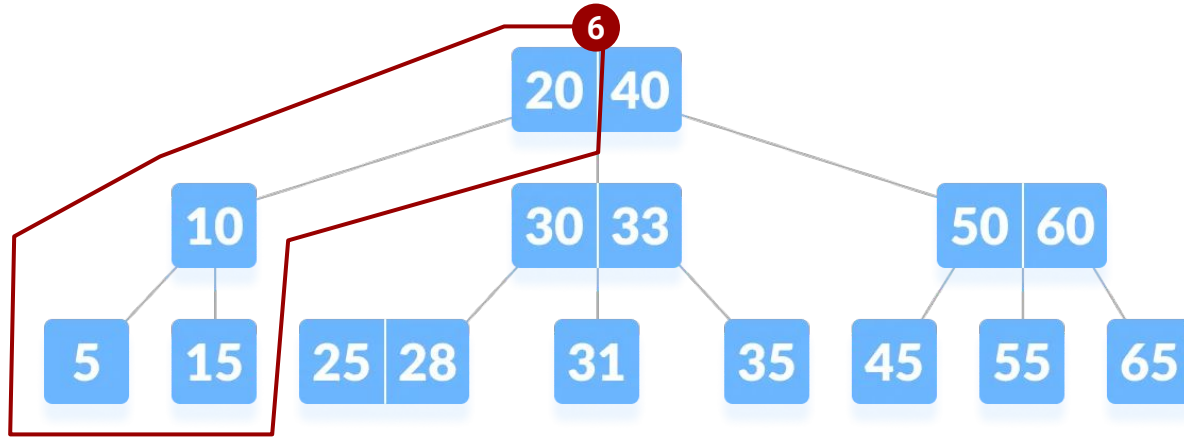
1. A node x can have n keys, where $x.n$ is the number of keys
2. If x is a node, $x.key_i$ is the i -th key of x
3. All n keys are sorted in non decreasing order: $x.key_1 \leq x.key_2 \leq x.key_n$

B-Tree: properties



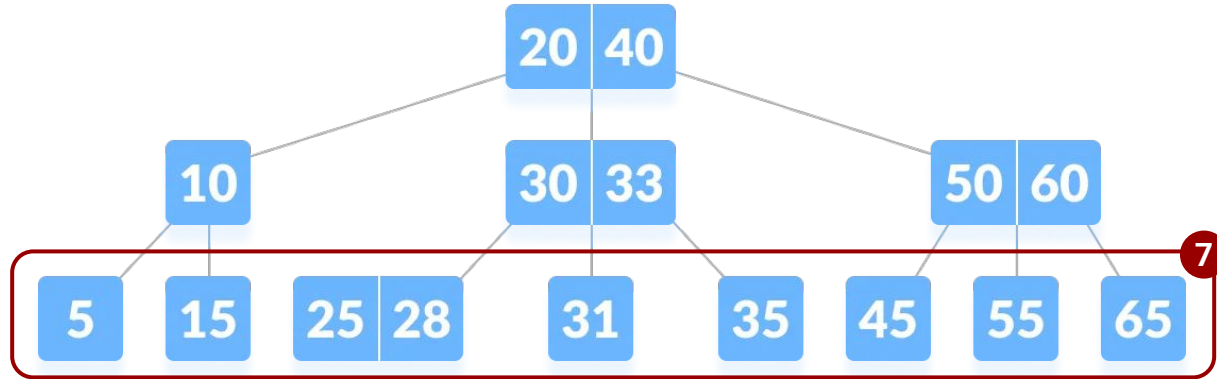
4. **x.leaf = true** if x is a leaf. If x is an internal node, **x.leaf = false**
5. Each internal node x contains **x.n+1** pointers, $x.c_1, x.c_2, \dots, x.c_{n+1}$ to its children

B-Tree: properties



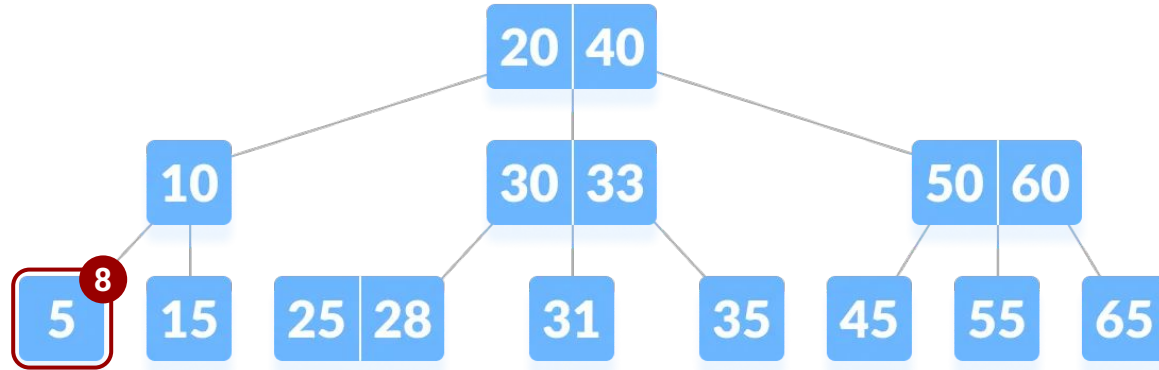
6. If x is a node, $x.\text{key}_i$ is the i -th key of x , all the children in $x.c_i$ are less than or equal to $x.\text{key}_i$

B-Tree: properties



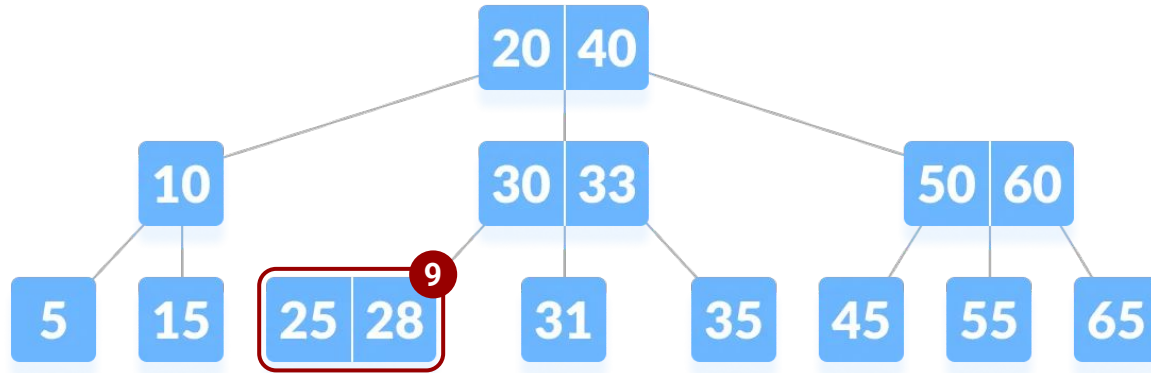
7. All leaves have the same depth, which is the tree's height h

B-Tree: properties



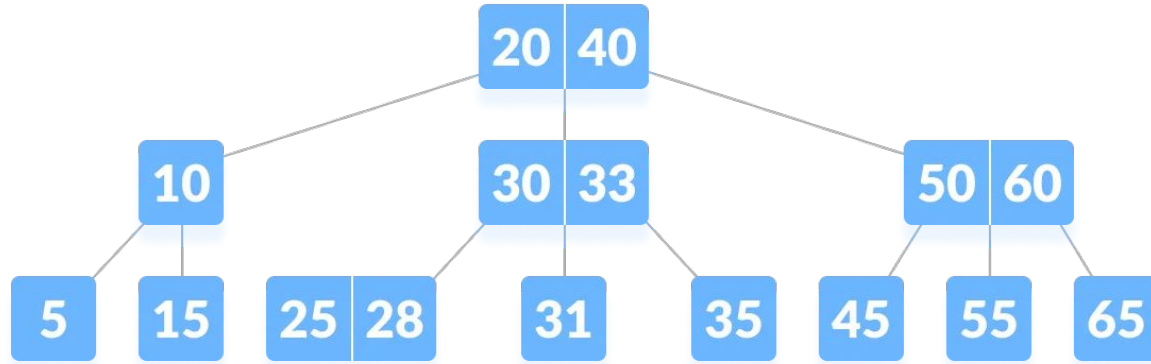
8. Let $t \geq 2$ be an integer, called **minimum degree** of the B-Tree, each node other than the root must have at least $t-1$ keys. The root must have at least one key

Properties of a B-Tree



9. Let $t \geq 2$ be an integer, called **minimum degree** of the B-Tree, each node may contain at most $2t-1$ keys. A node is **full** if it contains exactly $2t-1$ keys

Properties of a B-Tree



This B-Tree has a **minimum degree equals to $t = 2$** . Every internal node then has either 1 or 3 children. In practice, however, larger values of t yield B-trees with smaller height

Theorem

If the **number of keys** $n \geq 1$, for any n -key B-tree T of height h and minimum degree $t \geq 2$,

$$\log_{2t}(n + 1) - 1 \leq h \leq \log_t \frac{n + 1}{2}$$

Theorem: proof

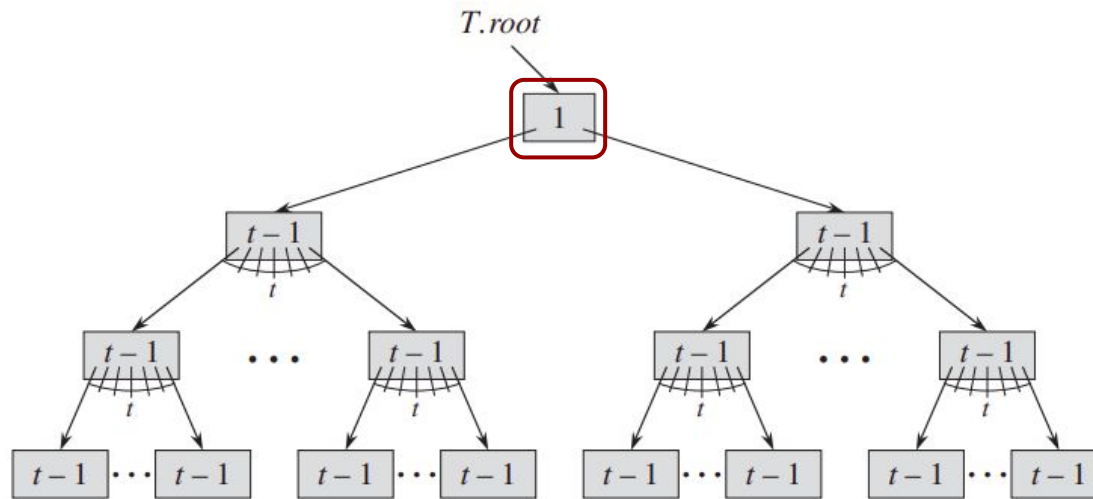
Let's prove that
$$h \leq \log_t \frac{n+1}{2}$$

According to the property 8, having a B-Tree with height ***h*** and a minimum degree equals to ***t***, the minimum number of keys is:

- one key for the root
- $t-1$ keys for the internal node.

See next picture

Theorem: proof

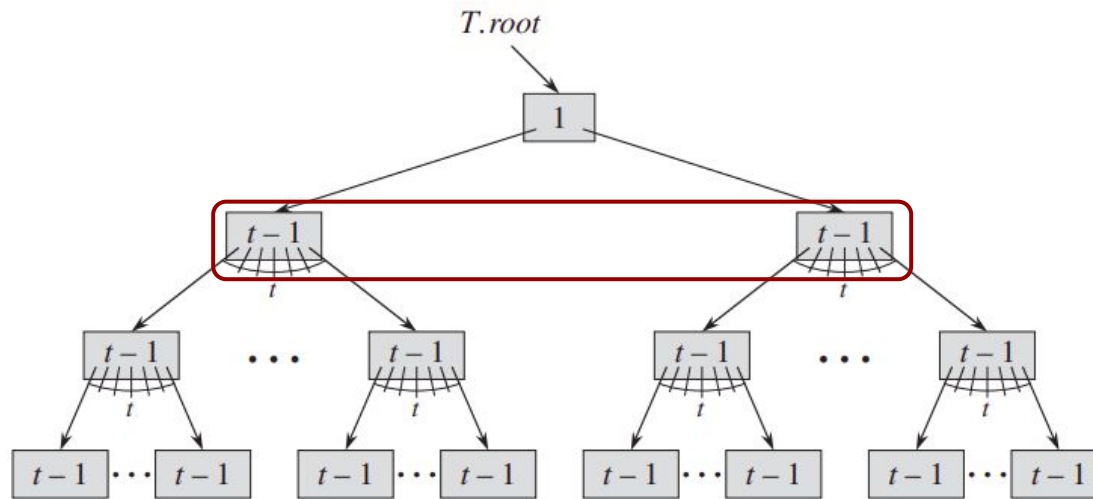


depth	number of nodes
0	1
1	2
2	$2t$
3	$2t^2$

$$n \geq 1 + \dots$$

total number of keys

Theorem: proof

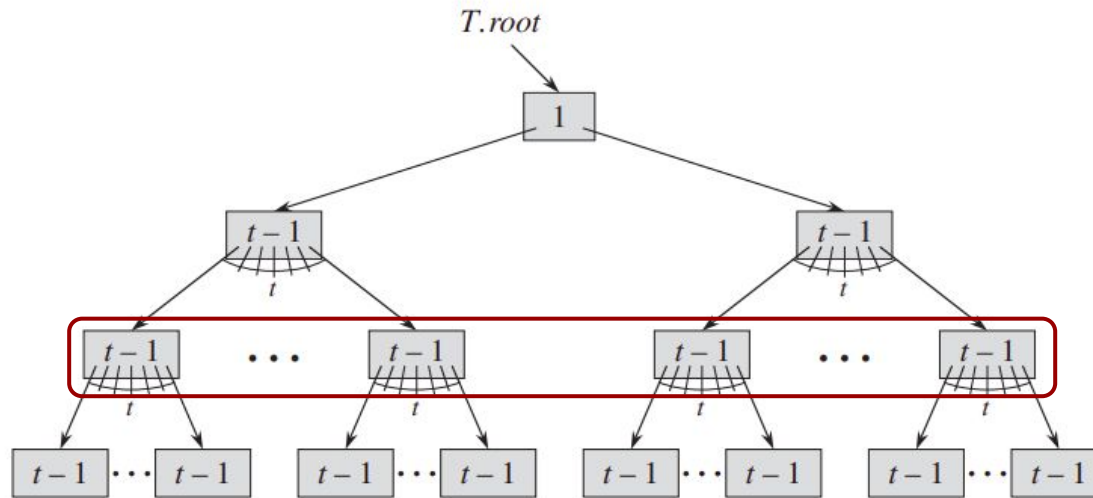


depth	number of nodes
0	1
1	2
2	$2t$
3	$2t^2$

$$n \geq 1 + 2(t-1) + \dots$$

total number of keys

Theorem: proof

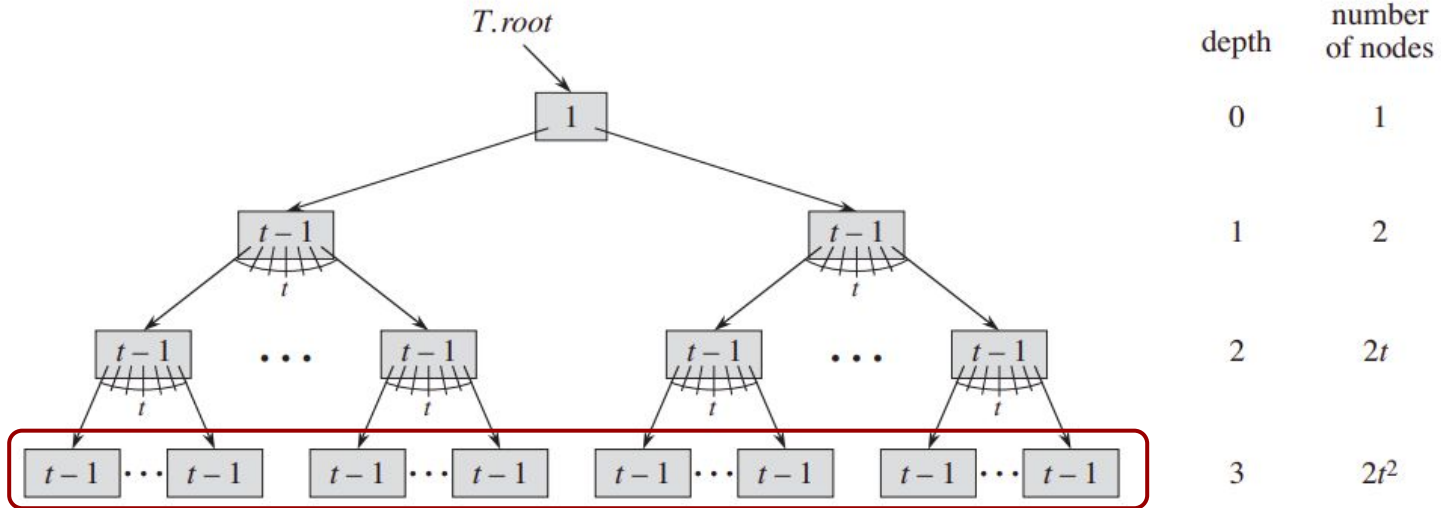


depth	number of nodes
0	1
1	2
2	$2t$
3	$2t^2$

$$n \geq 1 + 2(t-1) + \boxed{2t(t-1)} + \dots$$

total number of keys

Theorem: proof



$$n \geq 1 + 2(t - 1) + 2t(t - 1) + 2t^2(t + 1)$$

total number of keys

Theorem: proof

$$n \geq 1 + 2(t - 1) + 2t(t - 1) + 2t^2(t + 1)$$

$$n \geq 1 + \sum_{i=1}^h 2t^{i-1}(t - 1)$$

$$n \geq 1 + 2(t - 1) \sum_{i=1}^h t^{i-1}$$

Theorem: proof

$$n \geq 1 + 2(t - 1) \sum_{i=1}^h t^{i-1}$$

$$n \geq 1 + 2(t - 1) \left(\frac{t^h - 1}{t - 1} \right) = 1 + 2(t^h - 1)$$

Theorem: proof

$$n \geq 1 + 2(t^h - 1) = 2t^h - 1$$

$$n + 1 \geq 2t^h$$

$$\frac{n + 1}{2} \geq t^h$$

$$h \leq \log_t \frac{n + 1}{2}$$

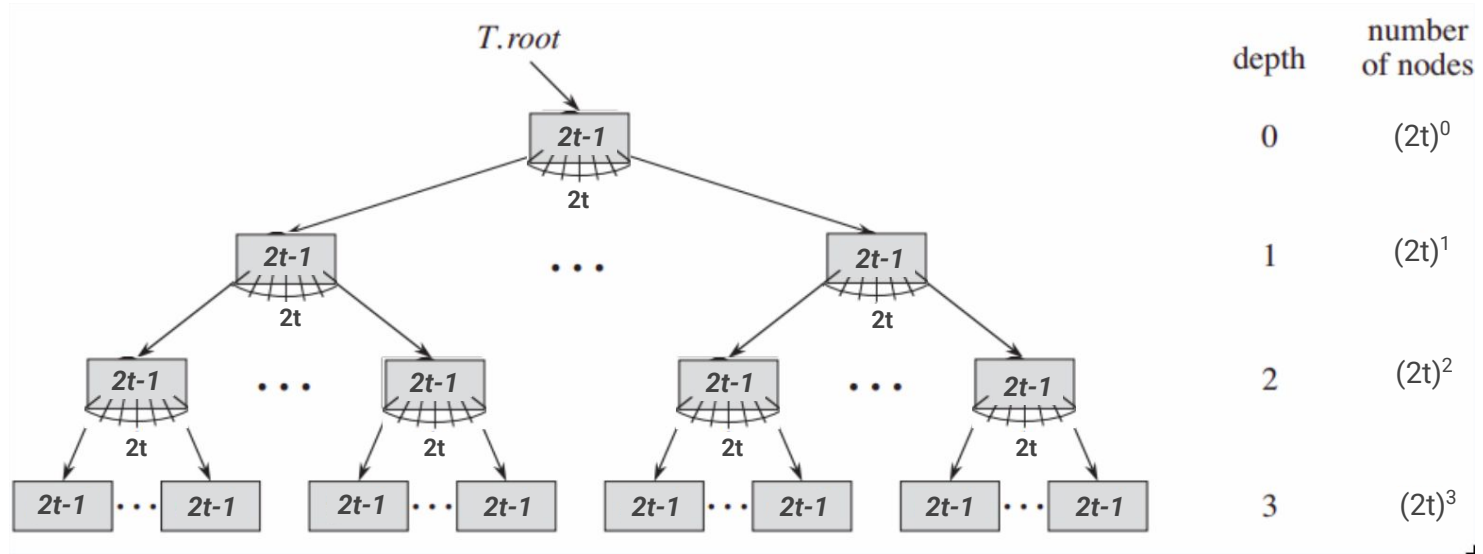
Theorem: proof

Let's prove that $\log_{2t}(n + 1) - 1 \leq h$

According to the property 9, having a B-Tree with height ***h*** and a minimum degree equals to ***t***, the maximum number of keys is $2t - 1$

See next picture

Theorem: proof



$$n \leq (2t - 1)(2t)^0 + (2t - 1)(2t)^1 + (2t - 1)(2t)^2 + (2t - 1)(2t)^3$$

number of keys

maximum number of keys per node

Theorem: proof

$$n \leq (2t-1)(2t)^0 + (2t-1)(2t)^1 + (2t-1)(2t)^2 + (2t-1)(2t)^3$$

$$n \leq (2t-1) \sum_{i=0}^h (2t)^i = (2t-1) \frac{(2t)^{h+1} - 1}{2t-1} = (2t)^{h+1} - 1$$

$$n \leq (2t)^{h+1} - 1$$

Theorem: proof

$$n \leq (2t)^{h+1} - 1$$

$$n + 1 \leq (2t)^{h+1}$$

$$\log_{2t}(n + 1) \leq h + 1$$

$$\log_{2t}(n + 1) - 1 \leq h$$

Creation Operation

B-TREE-CREATE(T)

```
1   $x = \text{ALLOCATE-NODE}()$ 
2   $x.\text{leaf} = \text{TRUE}$ 
3   $x.n = 0$ 
4   $\text{DISK-WRITE}(x)$ 
5   $T.\text{root} = x$ 
```

1. Create an empty node for the new tree T
2. Set the node a leaf
3. Set the number of keys to zero
4. Save the node on the disk
5. Set the root of the tree T

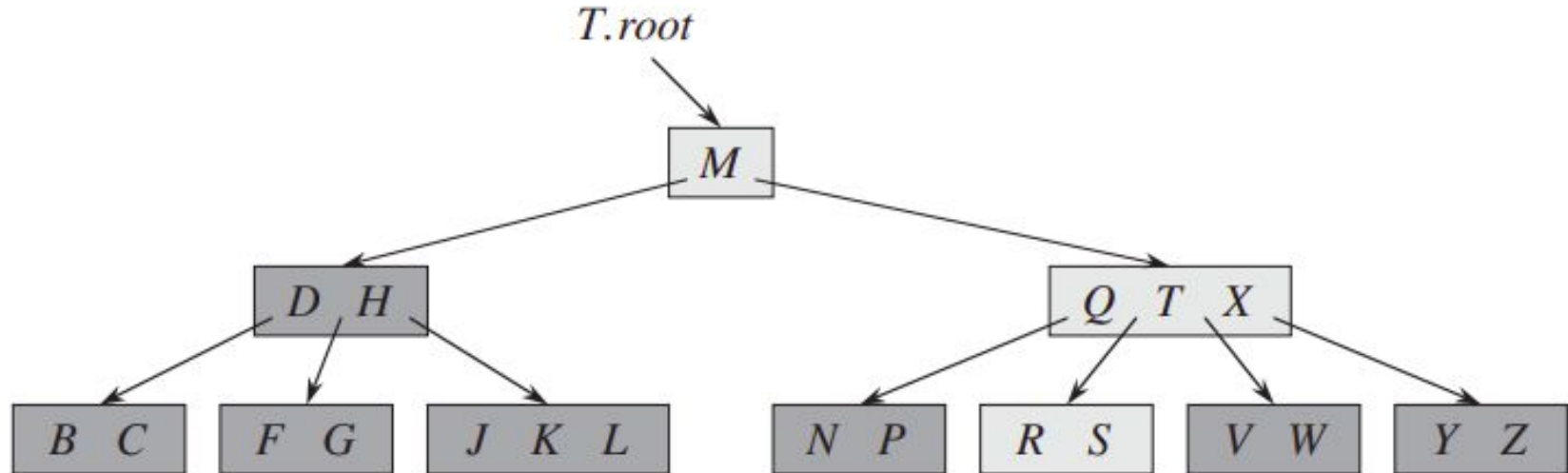
Creation Operation: complexity

B-TREE-CREATE(T)

```
1   $x = \text{ALLOCATE-NODE}()$ 
2   $x.\text{leaf} = \text{TRUE}$ 
3   $x.n = 0$ 
4   $\text{DISK-WRITE}(x)$ 
5   $T.\text{root} = x$ 
```

Execution Time = $O(1)$

Searching Operation for key = R



Searching Operation

B-TREE-SEARCH(x, k)

x : the root the the Tree

k : the key we are looking for

```
1   $i = 1$ 
2  while  $i \leq x.n$  and  $k > x.key_i$ 
3       $i = i + 1$ 
4  if  $i \leq x.n$  and  $k == x.key_i$ 
5      return  $(x, i)$ 
6  elseif  $x.leaf$ 
7      return NIL
8  else DISK-READ( $x.c_i$ )
9      return B-TREE-SEARCH( $x.c_i, k$ )
```

Searching Operation

B-TREE-SEARCH(x, k)

```
1   $i = 1$ 
2  while  $i \leq x.n$  and  $k > x.key_i$ 
3       $i = i + 1$ 
4  if  $i \leq x.n$  and  $k == x.key_i$ 
5      return  $(x, i)$ 
6  elseif  $x.leaf$ 
7      return NIL
8  else DISK-READ( $x.c_i$ )
9      return B-TREE-SEARCH( $x.c_i, k$ )
```

Scan all the keys in the root till we find a key greater than k (**linear scan**)

Searching Operation

B-TREE-SEARCH(x, k)

if we have found the key, just return it

```
1   $i = 1$ 
2  while  $i \leq x.n$  and  $k > x.key_i$ 
3       $i = i + 1$ 
4  if  $i \leq x.n$  and  $k == x.key_i$ 
5      return  $(x, i)$ 
6  elseif  $x.leaf$ 
7      return NIL
8  else DISK-READ( $x.c_i$ )
9      return B-TREE-SEARCH( $x.c_i, k$ )
```


Searching Operation

B-TREE-SEARCH(x, k)

```
1   $i = 1$ 
2  while  $i \leq x.n$  and  $k > x.key_i$ 
3       $i = i + 1$ 
4  if  $i \leq x.n$  and  $k == x.key_i$ 
5      return  $(x, i)$ 
6  elseif  $x.leaf$ 
7      return NIL
8  else DISK-READ( $x.c_i$ )
9      return B-TREE-SEARCH( $x.c_i, k$ )
```

If we haven't found yet the key, and the root is a leaf, then give up

Searching Operation

B-TREE-SEARCH(x, k)

```
1   $i = 1$ 
2  while  $i \leq x.n$  and  $k > x.key_i$ 
3       $i = i + 1$ 
4  if  $i \leq x.n$  and  $k == x.key_i$ 
5      return  $(x, i)$ 
6  elseif  $x.leaf$ 
7      return NIL
8  else DISK-READ( $x.c_i$ )
9      return B-TREE-SEARCH( $x.c_i, k$ )
```

If we haven't found yet the key, and the root is not a leaf, search for the key if the subtree (**recursive call**)

Searching Operation: complexity

B-TREE-SEARCH(x, k)

```
1   $i = 1$ 
2  while  $i \leq x.n$  and  $k > x.key_i$ 
3       $i = i + 1$ 
4  if  $i \leq x.n$  and  $k == x.key_i$ 
5      return  $(x, i)$ 
6  elseif  $x.leaf$ 
7      return NIL
8  else DISK-READ( $x.c_i$ )
9  return B-TREE-SEARCH( $x.c_i, k$ )
```

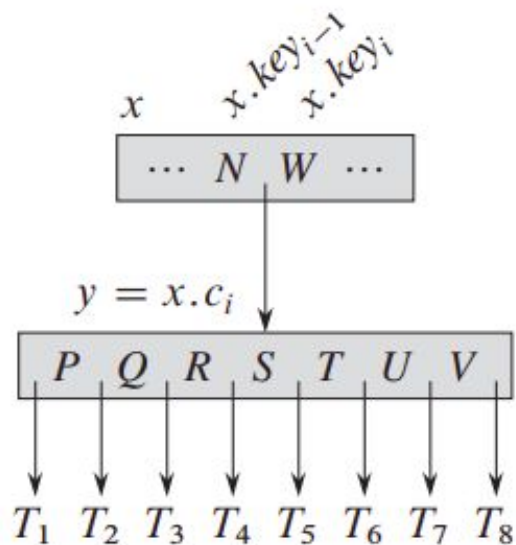
1. The recursive B-Tree-Search function is invoked $\log_t n$
2. Every time the function is invoked, the while cycle is executed $2t$ times the most

Execution time = $O(t \log_t n)$

Splitting Operation

- It is invoked when **a node is full** (the number of keys is $2t - 1$)
- During Splitting Operation, a full node is split in two nodes having $t - 1$ keys each
- After its execution, **the B-Tree grows in height by one**

Splitting Operation

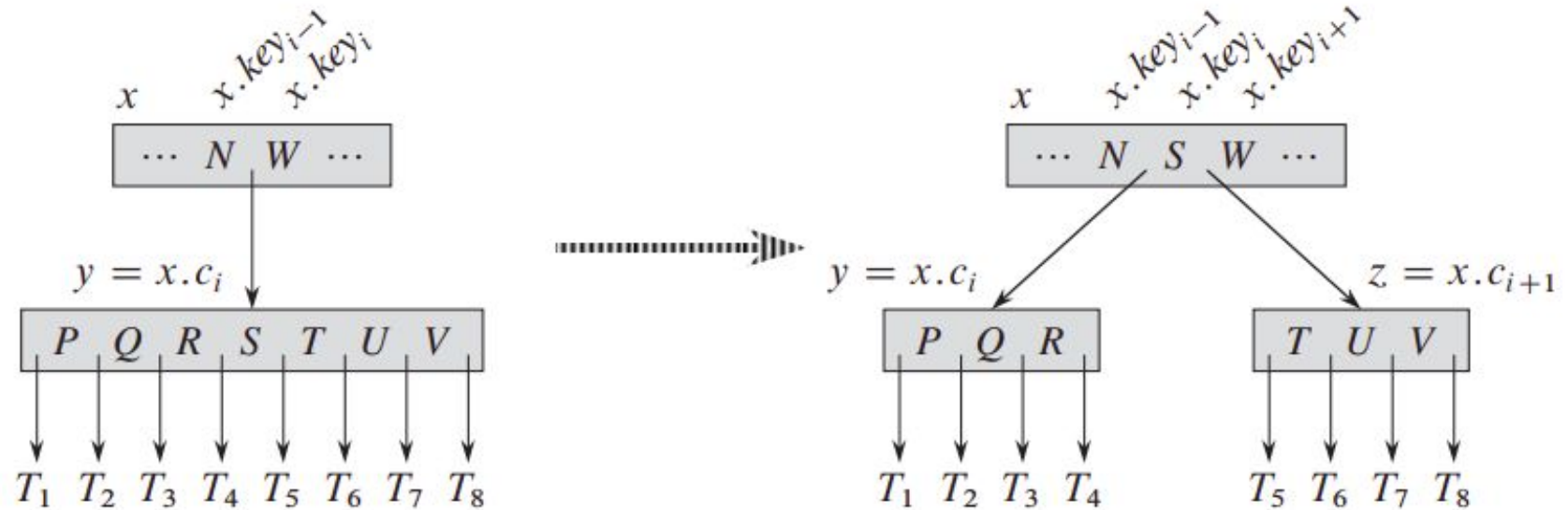


In this example $t = 4$, then a node is full when the number of keys is $2t - 1 = 7$

The node y is full, x instead is a **non-full** node

The key S is the median element in y

Splitting Operation



Splitting Operation

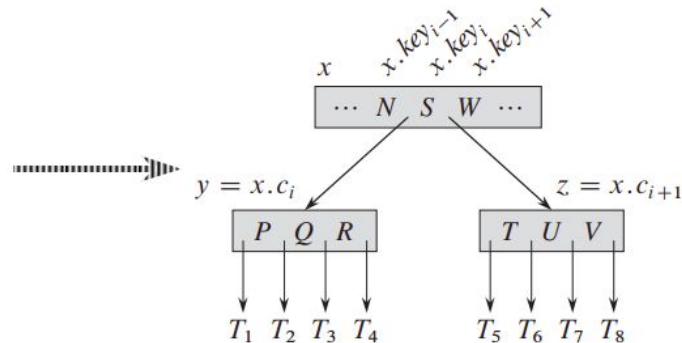
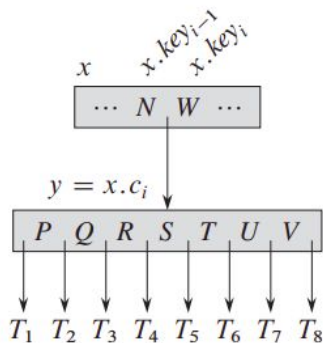
B-TREE-SPLIT-CHILD(x, i)

```
1   $z = \text{ALLOCATE-NODE}()$ 
2   $y = x.c_i$ 
3   $z.\text{leaf} = y.\text{leaf}$ 
4   $z.n = t - 1$ 
5  for  $j = 1$  to  $t - 1$ 
6       $z.\text{key}_j = y.\text{key}_{j+t}$ 
7  if not  $y.\text{leaf}$ 
8      for  $j = 1$  to  $t$ 
9           $z.c_j = y.c_{j+t}$ 
10  $y.n = t - 1$ 
```

x is a nonfull internal node

i is the index such that $x.c_i$ is a full child of x

1. node z is created and initialized

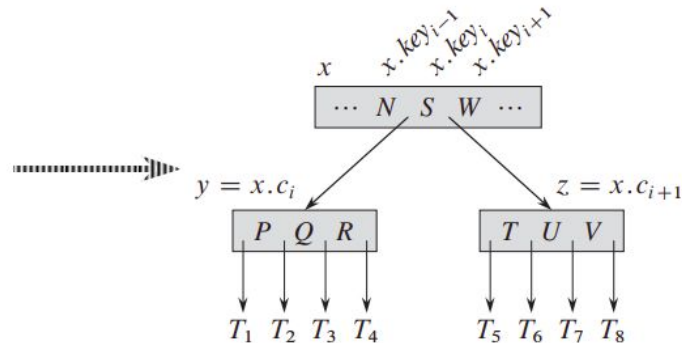
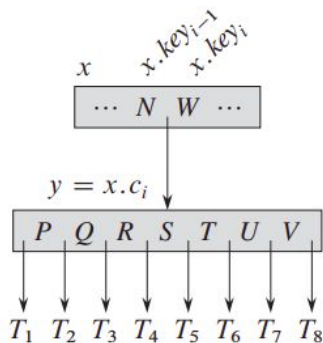


Splitting Operation

B-TREE-SPLIT-CHILD(x, i)

```
1   $z = \text{ALLOCATE-NODE}()$ 
2   $y = x.c_i$ 
3   $z.\text{leaf} = y.\text{leaf}$ 
4   $z.n = t - 1$ 
5  for  $j = 1$  to  $t - 1$ 
6       $z.\text{key}_j = y.\text{key}_{j+t}$ 
7  if not  $y.\text{leaf}$ 
8      for  $j = 1$  to  $t$ 
9           $z.c_j = y.c_{j+t}$ 
10  $y.n = t - 1$ 
```

2. All the keys in z are copied from y
3. All the links to the z 's children are copied from y
4. Adjusts the key count for y



Splitting Operation

```

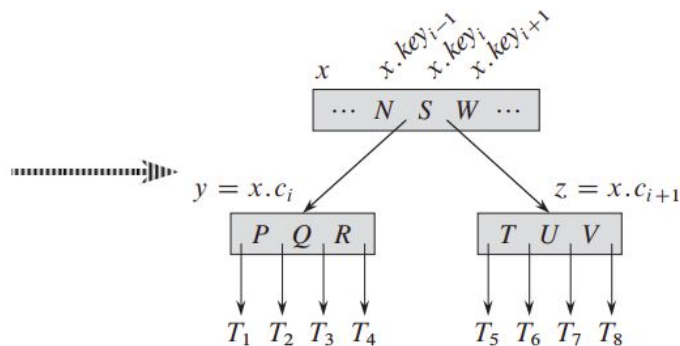
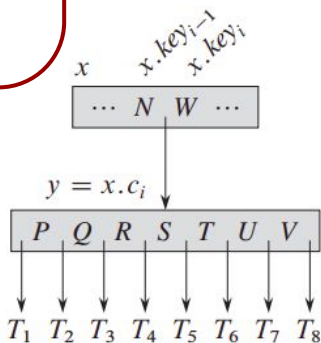
11  for  $j = x.n + 1$  downto  $i + 1$ 
12       $x.c_{j+1} = x.c_j$ 
13   $x.c_{i+1} = z$ 
14  for  $j = x.n$  downto  $i$ 
15       $x.key_{j+1} = x.key_j$ 
16   $x.key_i = y.key_t$ 
17   $x.n = x.n + 1$ 
    
```

```

18  DISK-WRITE( $y$ )
19  DISK-WRITE( $z$ )
20  DISK-WRITE( $x$ )
    
```

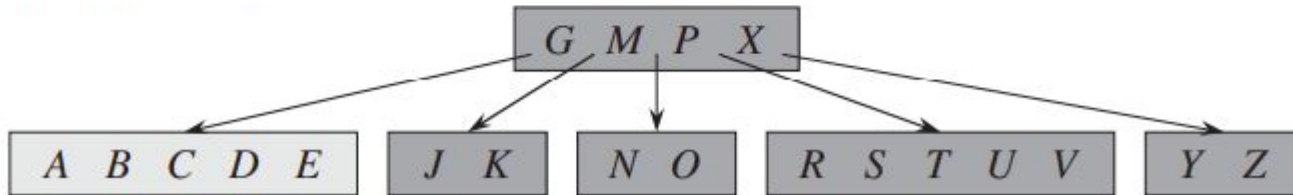
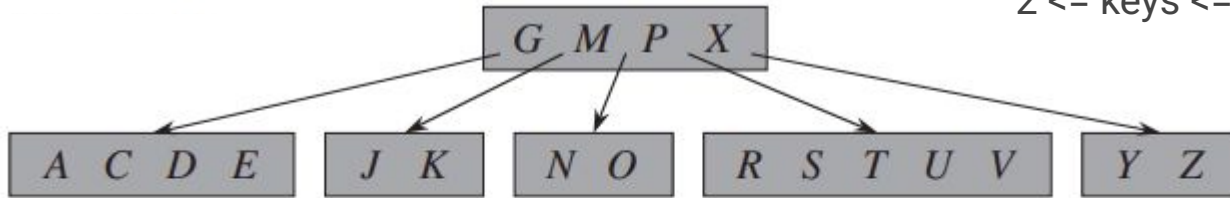
5. Insert z as a child of x , move the median key from y up to x
6. Save permanently all the modified node

Complexity = $O(t)$



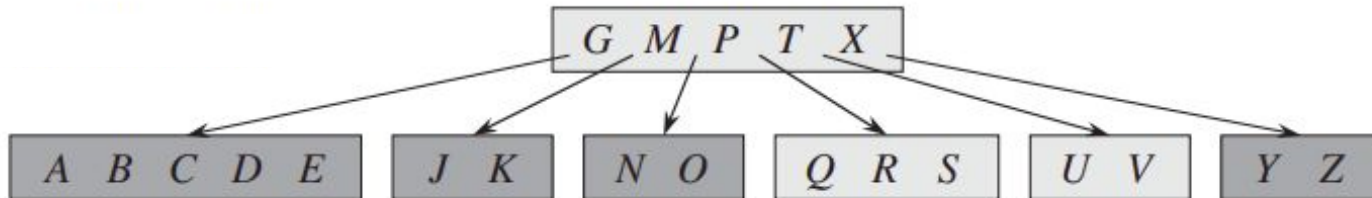
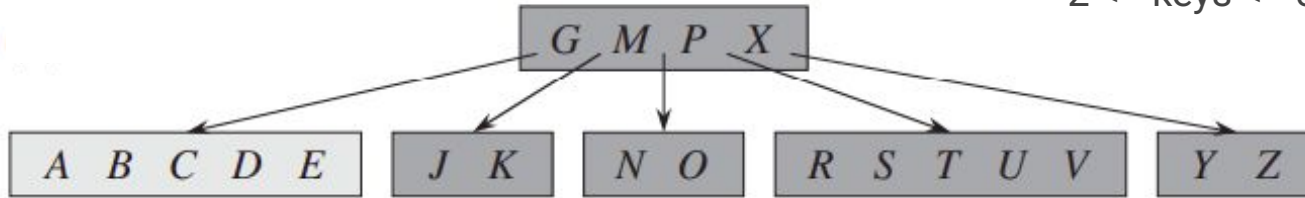
Inserting Operation: inserting B

$t = 3$
 $2 \leq \text{keys} \leq 5$

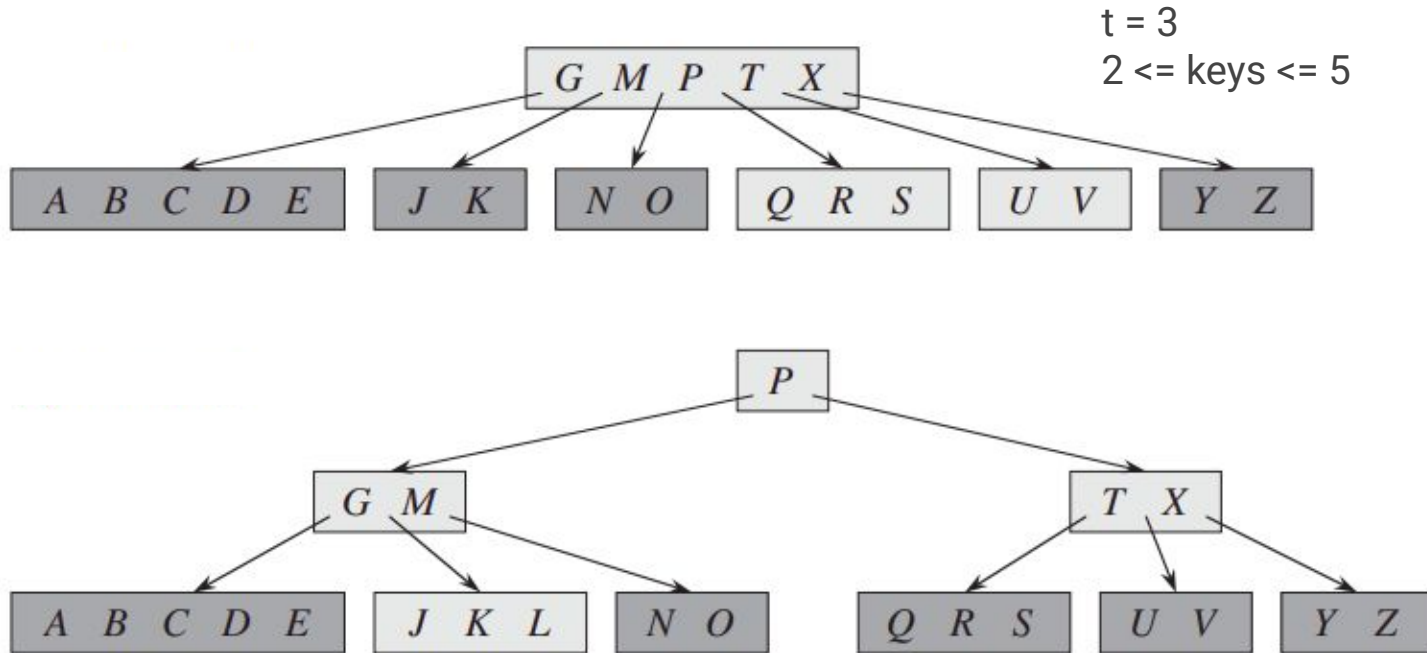


Inserting Operation: inserting Q

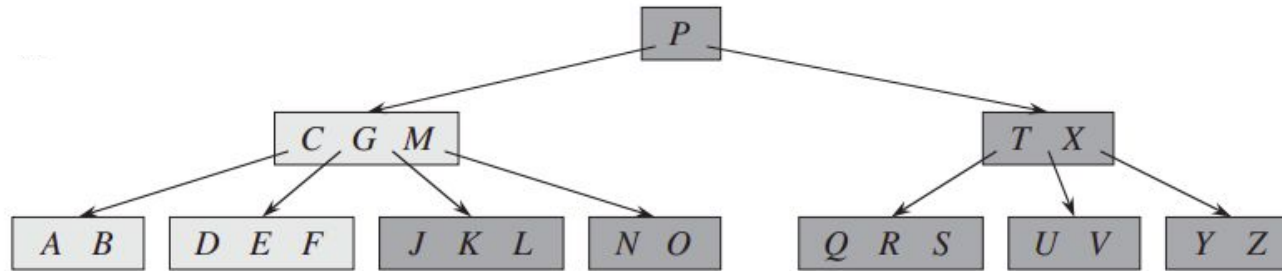
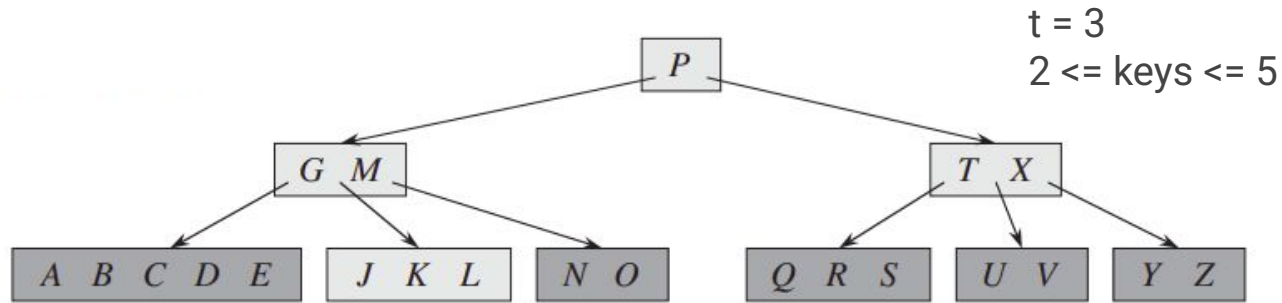
$t = 3$
 $2 \leq \text{keys} \leq 5$



Inserting Operation: inserting L



Inserting Operation: inserting F



Insertion Operation

B-TREE-INSERT(T, k)

```
1   $r = T.root$ 
2  if  $r.n == 2t - 1$ 
3       $s = \text{ALLOCATE-NODE}()$ 
4       $T.root = s$ 
5       $s.leaf = \text{FALSE}$ 
6       $s.n = 0$ 
7       $s.c_1 = r$ 
8  B-TREE-SPLIT-CHILD( $s, 1$ )
9  B-TREE-INSERT-NONFULL( $s, k$ )
10 else B-TREE-INSERT-NONFULL( $r, k$ )
```

T is the B Tree, k is the key to be inserted

1. If the root is full, the root is split, a new node s becomes the root. So, s is a node without any key ($s.n = 0$), with only one pointer pointing to r (the old Tree Root)
2. The first (and unique) child of s is split. This increases the height of a B-tree

Insertion Operation

B-TREE-INSERT(T, k)

```
1   $r = T.root$ 
2  if  $r.n == 2t - 1$ 
3       $s = \text{ALLOCATE-NODE}()$ 
4       $T.root = s$ 
5       $s.leaf = \text{FALSE}$ 
6       $s.n = 0$ 
7       $s.c_1 = r$ 
8      B-TREE-SPLIT-CHILD( $s, 1$ )
9      B-TREE-INSERT-NONFULL( $s, k$ )
10 else B-TREE-INSERT-NONFULL( $r, k$ )
```

3. If the original root is full, call the Tree-Insert-NonFull procedure on s
4. Otherwise, call the procedure on r

Insertion Operation

B-TREE-INSERT-NONFULL(x, k)

```
1   $i = x.n$ 
2  if  $x.leaf$ 
3      while  $i \geq 1$  and  $k < x.key_i$ 
4           $x.key_{i+1} = x.key_i$ 
5           $i = i - 1$ 
6       $x.key_{i+1} = k$ 
7       $x.n = x.n + 1$ 
8      DISK-WRITE( $x$ )
```

1. We are inserting k in a leaf (all the keys are pushed ahead to create the proper space for storing k). The number of keys is increased by 1

Insertion Operation

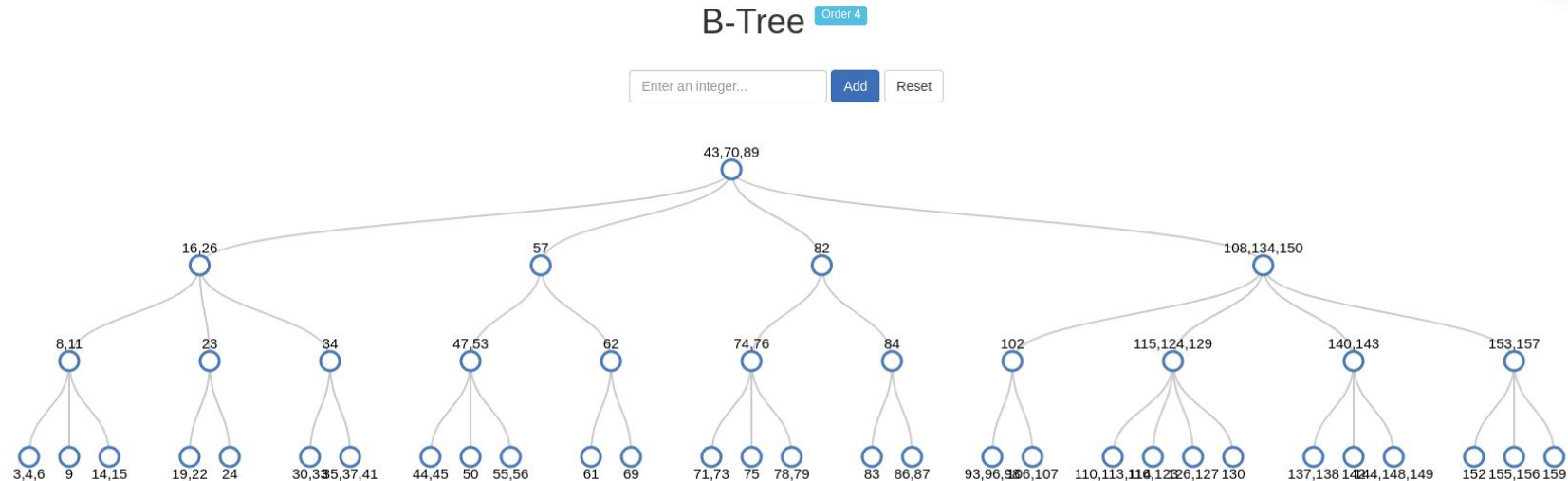
```
9  else while  $i \geq 1$  and  $k < x.key_i$ 
10       $i = i - 1$ 
11       $i = i + 1$ 
12      DISK-READ( $x.c_i$ )
13      if  $x.c_i.n == 2t - 1$ 
14          B-TREE-SPLIT-CHILD( $x, i$ )
15          if  $k > x.key_i$ 
16               $i = i + 1$ 
17      B-TREE-INSERT-NONFULL( $x.c_i, k$ )
```

2. The node is not a leaf, so we need to understand in which subtree k must be added
3. Once we have found the right subtree, we verify whether or not such node is full (and then must be split out)
4. k is then inserted in the right position

Complexity: $O(t \log_t n)$

B-Tree Simulator

<https://yangez.github.io/btree-js/>



B-Tree Simulator

<https://www.cs.usfca.edu/~galles/visualization/BTree.html>

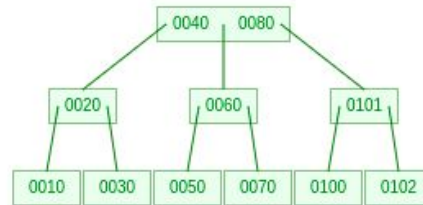
☒ Max. Degree = 3 ☐ Preemptive Split / Merge (Even max degree only)

☐ Max. Degree = 4

☐ Max. Degree = 5

☐ Max. Degree = 6

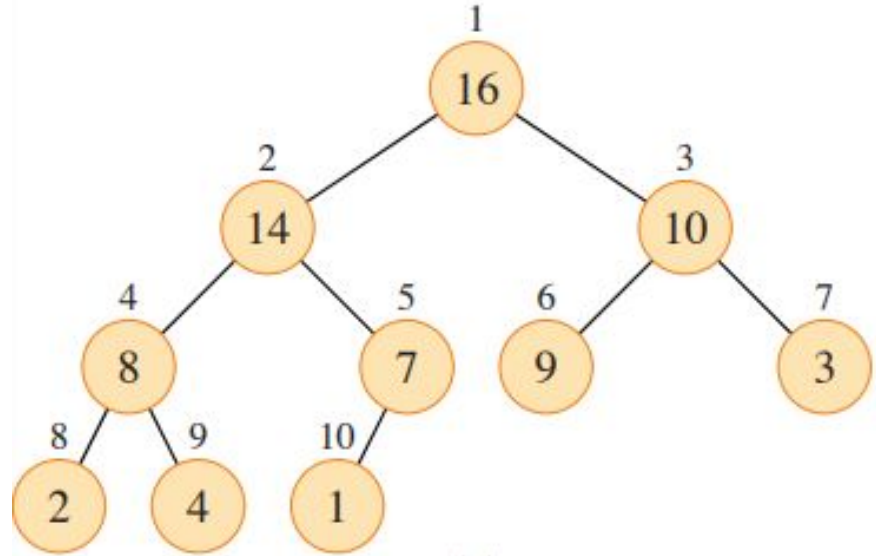
☐ Max. Degree = 7



Binary Heap

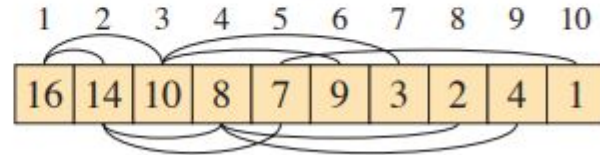
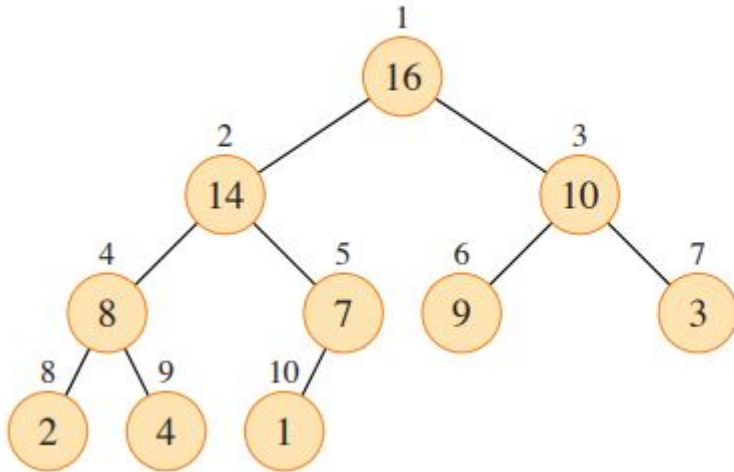
Binary Heap: properties

1. It is a **nearly complete** binary tree
2. It is completely filled on all levels except possibly the lowest, which is filled **from the left up to a point**



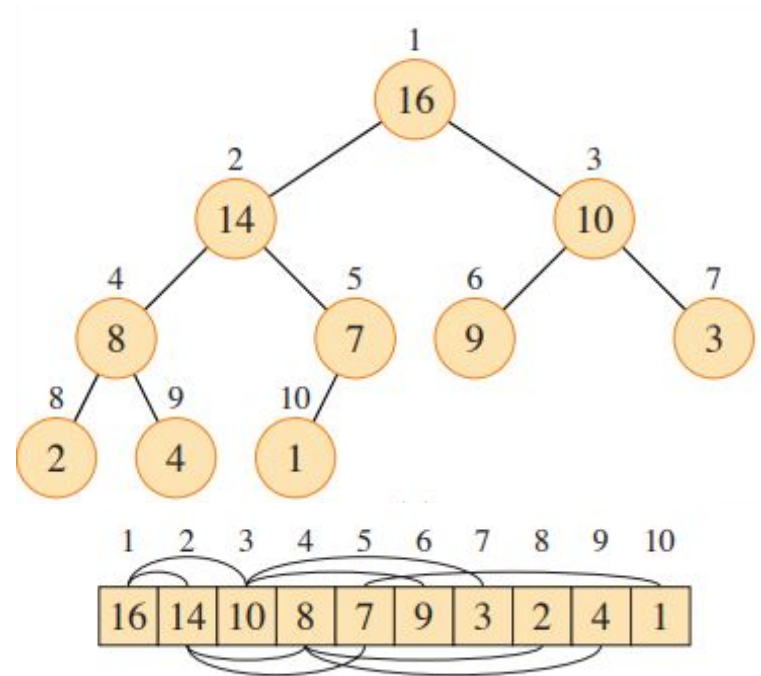
Binary Heap

From the implementation point of view, a Binary Heap is implemented with an array



Binary Heap: basic operation

- Let **A** be an array representing a Binary Heap
- **A.heap-size** is the number of elements in the Heap
- Let **i** be the index of an element (from **1** to **heap-size**)
- **A[1]** is the root of the heap



Binary Heap: basic operation

PARENT(i)

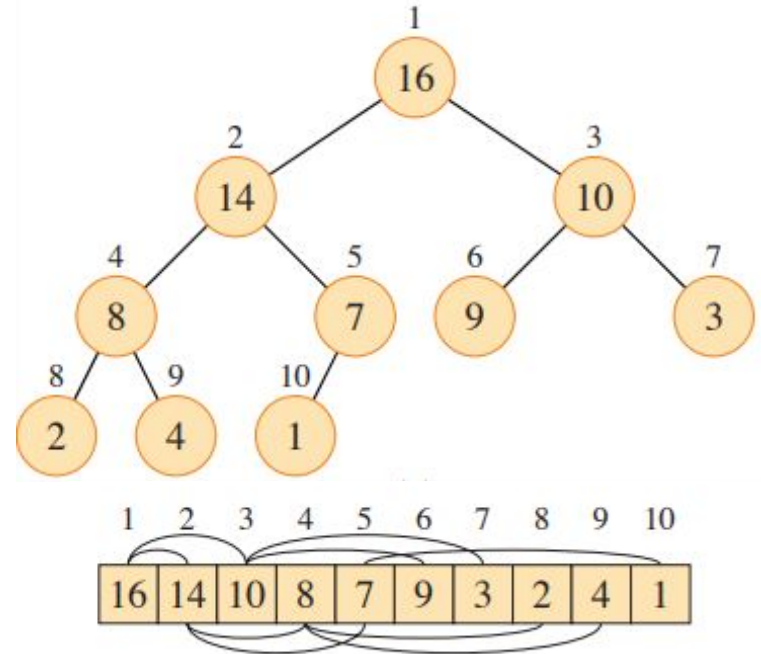
1 **return** $\lfloor i/2 \rfloor$

LEFT(i)

1 **return** $2i$

RIGHT(i)

1 **return** $2i + 1$



Binary Heap: basic operation - Complexity

Parent(i) $O(1)$

Left(i) $O(1)$

Right(i) $O(1)$

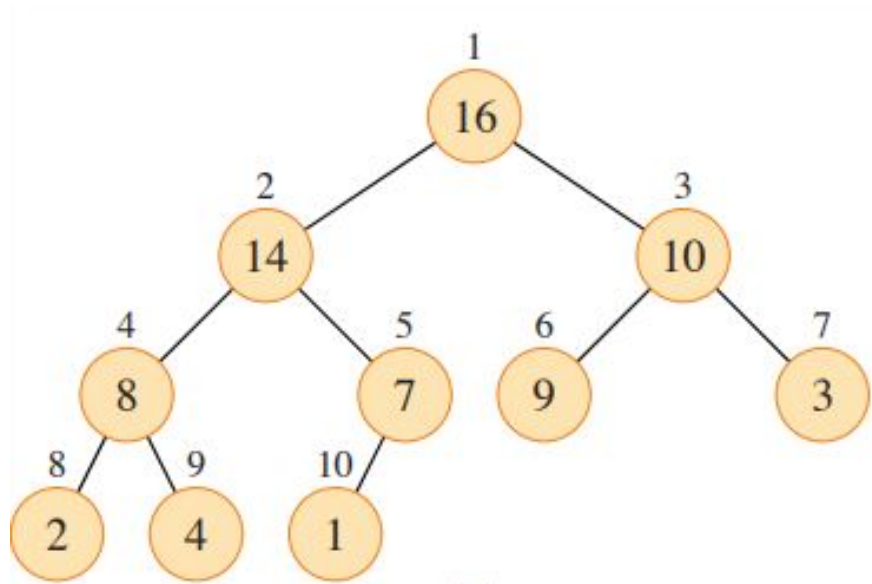
Binary Heap: max-heap and min-heap

There are two kinds of Binary Heap:

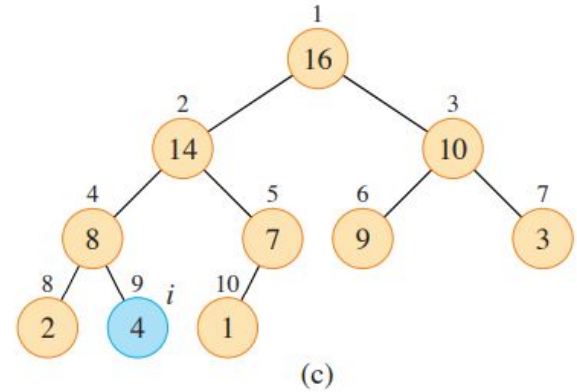
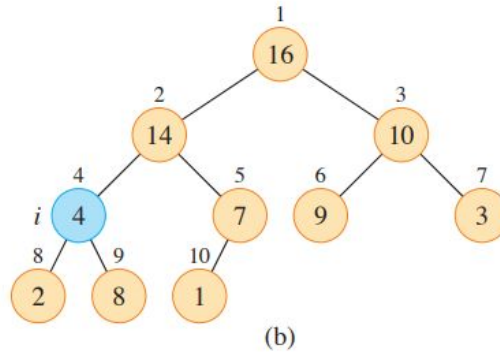
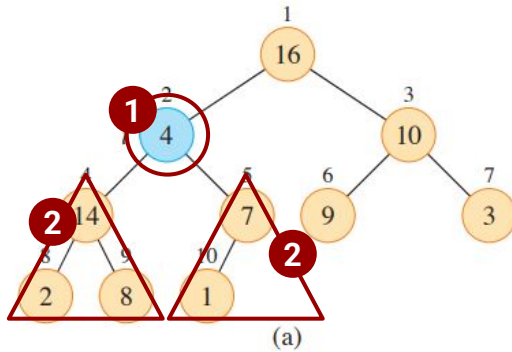
- max-heap
 - $A[\text{Parent}(i)] \geq A[i]$
- min-heap
 - $A[\text{Parent}(i)] \leq A[i]$

In a max-heap, $A[1]$ contains the maximum value; in a min-heap, $A[1]$ contains the minimum value

We will focus on **max-heap**

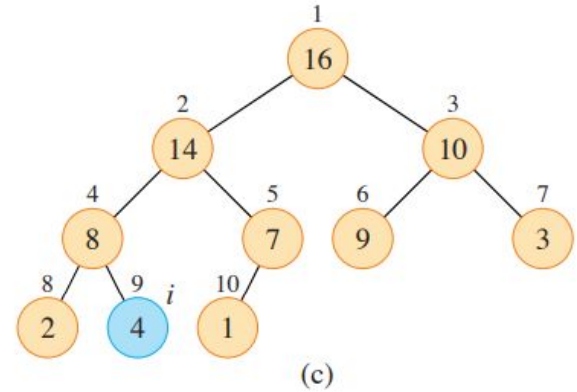
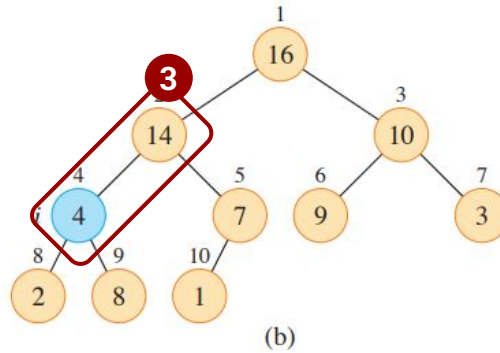
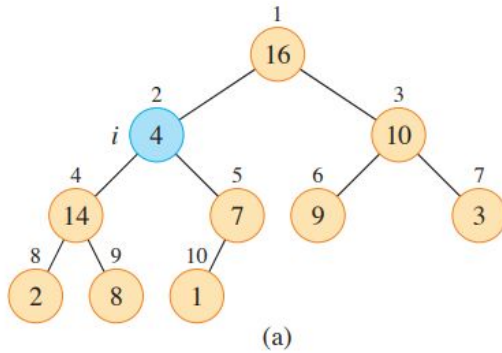


Binary Heap: max-heapify



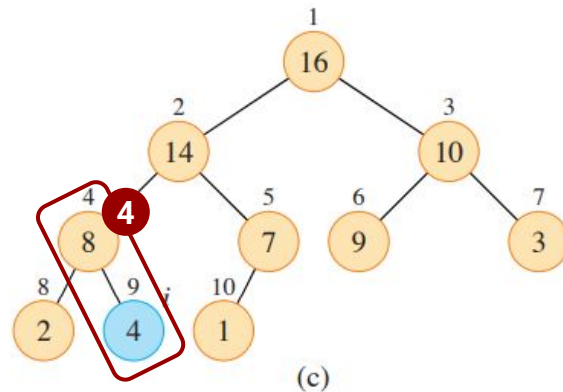
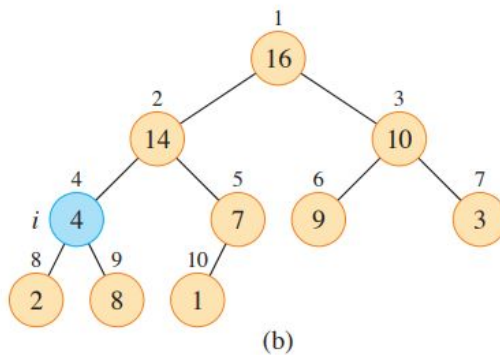
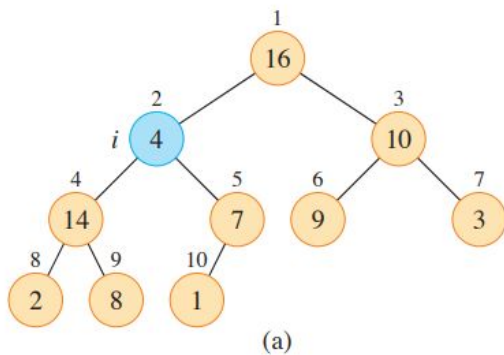
1. We want to run max-heapify on $i=2$ ($A[2] = 4$);
2. $\text{Left}(i)$ and $\text{Right}(i)$ are two Binary Max-Heap

Binary Heap: max-heapify



3. We compare $A[1]$ with $A[\text{Left}(i)]$ and $A[\text{Right}(i)]$ and the maximum value is moved on the root

Binary Heap: max-heapify



3. We run max-heapify recursively only on the $i = 4$ (which is the one which destroys the max-heap property). Then $A[4]$ and $A[9]$ are swapped

Binary Heap: max-heapify

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

Recursive function

Complexity:

$O(\log_2 n)$

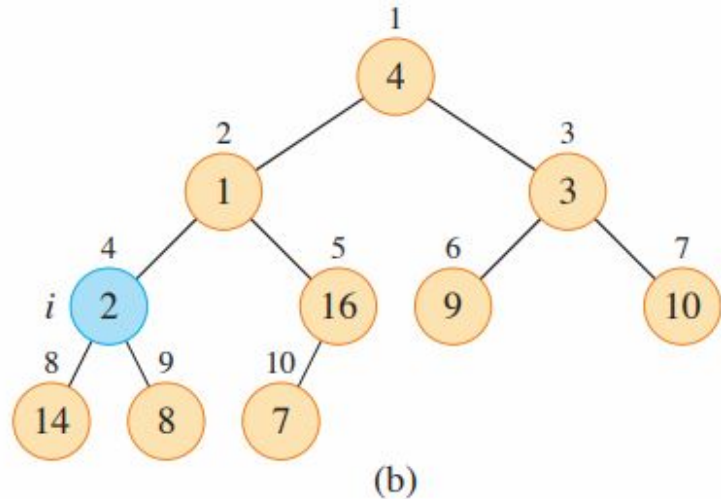
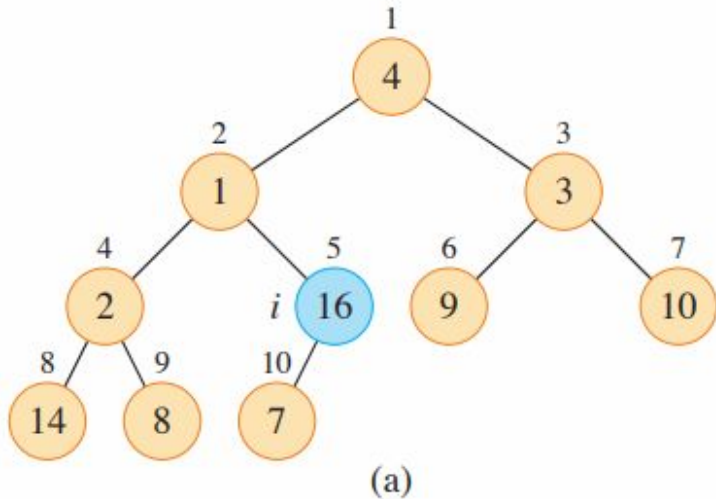
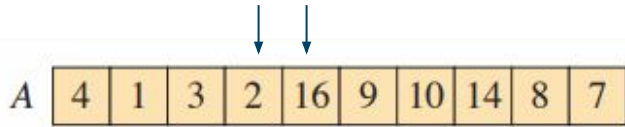
Binary max-heap: build max heap

Let's suppose we have an array which is not a Max-Heap and we would like to turn the array into a max-heap

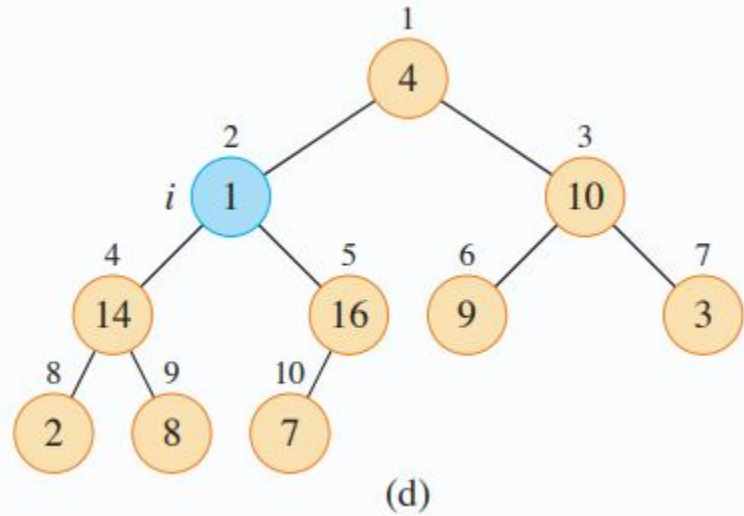
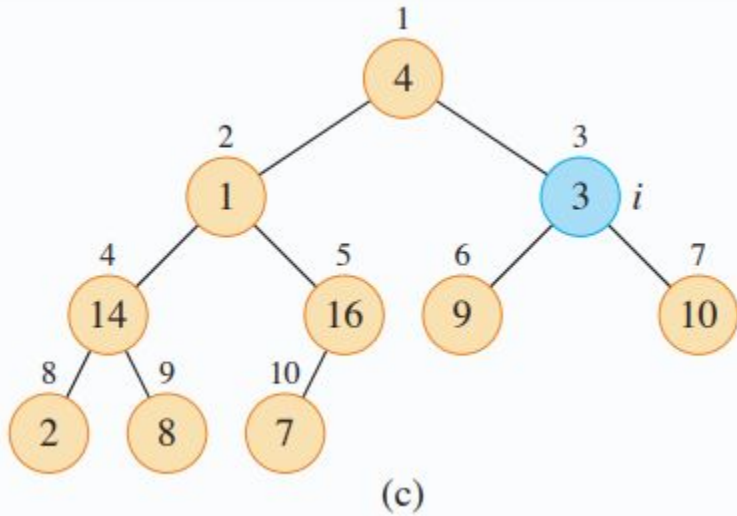
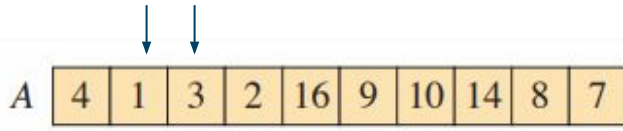
A	4	1	3	2	16	9	10	14	8	7
---	---	---	---	---	----	---	----	----	---	---

We start from the center of the array, which is $\text{floor}(n/2)$, and then run the heapify operation on all elements **from $\text{floor}(n/2)$ to 1**

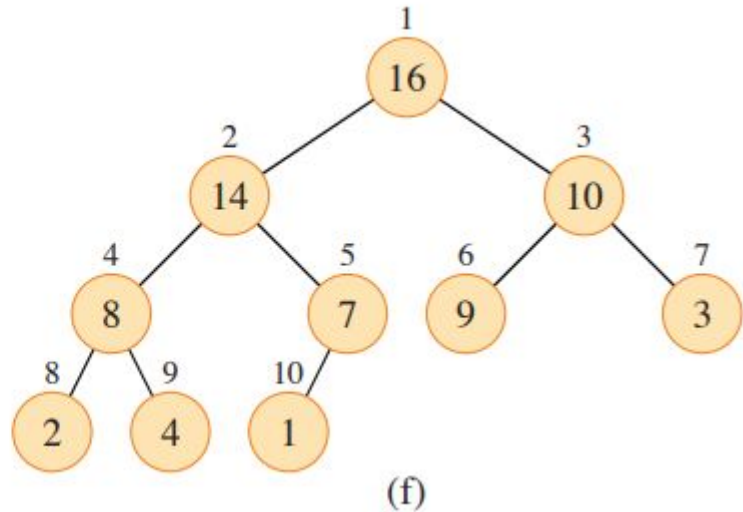
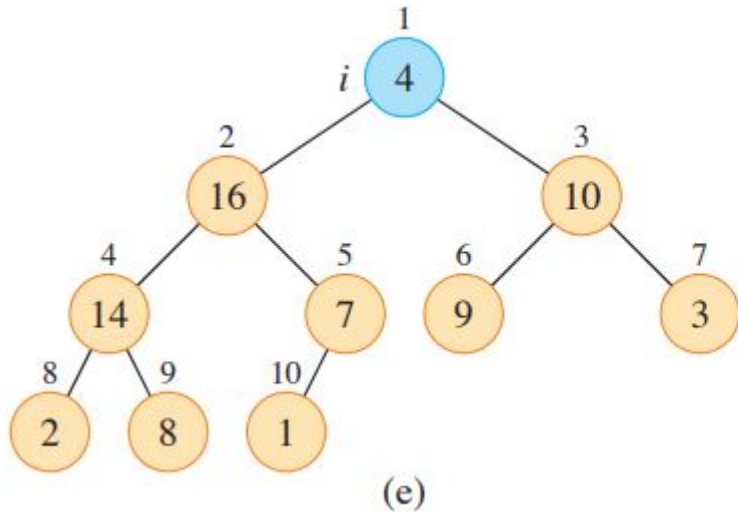
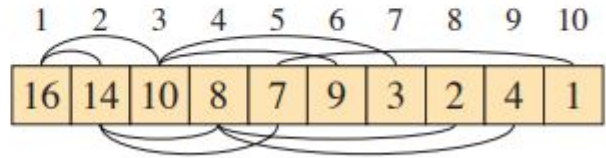
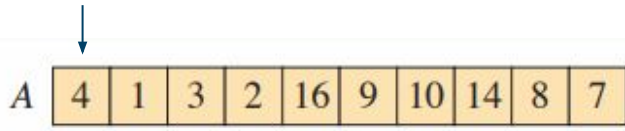
Binary max-heap: build max heap



Binary max-heap: build max heap



Binary max-heap: build max heap



Binary max-heap: build max heap - Complexity

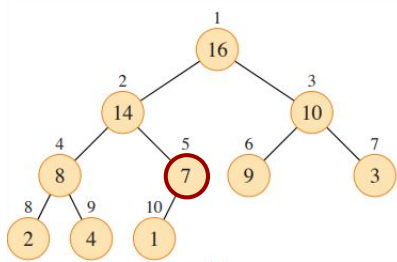
we can build a max-heap from an unordered array in linear time:

$$O(n)$$

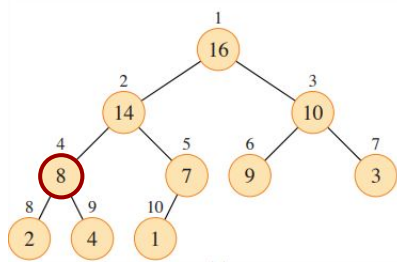
Proof:

1. we start from the middle of the array (from the last non-leaf node) and we move backward till the first element of the array. Total number: $\lfloor n/2 \rfloor$
2. for each element we perform the heapify operation, which takes $O(h)$ where h is the height of the subtree at the current node

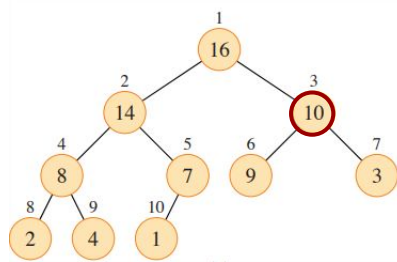
Binary max-heap: build max heap - Complexity



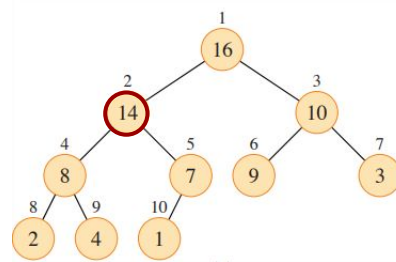
$$h_5 = 1 = \lfloor \log_2 10 \rfloor - \lfloor \log_2 5 \rfloor$$



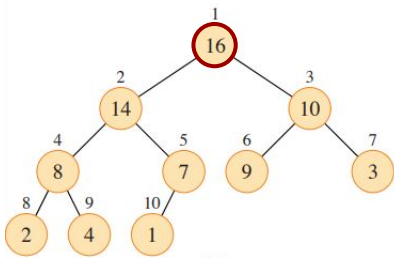
$$h_4 = 1 = \lfloor \log_2 10 \rfloor - \lfloor \log_2 4 \rfloor$$



$$h_3 = 1 = \lfloor \log_2 10 \rfloor - \lfloor \log_2 3 \rfloor$$



$$h_2 = 2 = \lfloor \log_2 10 \rfloor - \lfloor \log_2 2 \rfloor$$



$$h_1 = 2 = \lfloor \log_2 10 \rfloor - \lfloor \log_2 1 \rfloor$$

$$T(n) = \sum_{i=1}^{\lfloor n/2 \rfloor} \lfloor \log_2 n \rfloor - \lfloor \log_2 i \rfloor$$

Binary max-heap: build max heap - Complexity

$$T(n) = \sum_{i=1}^{\lfloor n/2 \rfloor} \lfloor \log_2 n \rfloor - \lfloor \log_2 i \rfloor \leq \sum_{i=1}^{n/2} \log_2 n - \sum_{i=1}^{n/2} \log_2 i = \frac{n}{2} \log_2 n - \log_2 \prod_{i=1}^{n/2} i$$

$$T(n) \leq \frac{n}{2} \log_2 n - \log_2 \frac{n}{2}! = \frac{n}{2} \log_2 \frac{2n}{2} - \log_2 \frac{n}{2}! = \frac{n}{2} (\log_2 \frac{n}{2} + \log_2 2) - \log_2 \frac{n}{2}!$$

$$T(n) \leq \frac{n}{2} \log_2 n - \log_2 \frac{n}{2}! = \frac{n}{2} \log_2 \frac{2n}{2} - \log_2 \frac{n}{2}! = \frac{n}{2} (\log_2 \frac{n}{2} + \log_2 2) - \log_2 \frac{n}{2}!$$

Stirling's approximation
 $\log_e n! \approx n \log_e n - n$

$$T(n) \leq \frac{n}{2} \log_2 \frac{n}{2} + \frac{n}{2} - \frac{n}{2} \log_2 \frac{n}{2} + \frac{n}{2} \log_2 e = \frac{n}{2} (1 + \log_2 e) = O(n)$$

Binary max-heap: max-heap-insert

Insert(A, 15) A

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

A

1	2	3	4	5	6	7	8	9	10	$i=11$
16	14	10	8	7	9	3	2	4	1	15

$A[\text{parent}[i]] < A[i]$

A

1	2	3	4	$i=5$	6	7	8	9	10	11
16	14	10	8	15	9	3	2	4	1	7

$A[\text{parent}[i]] < A[i]$

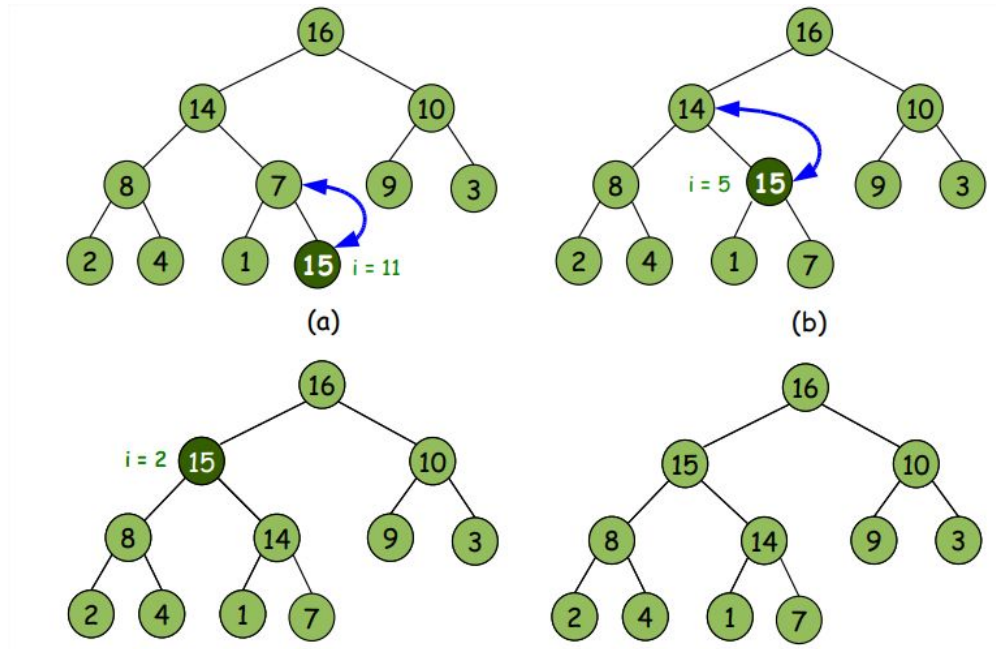
A

1	$i=2$	3	4	5	6	7	8	9	10	11
16	15	10	8	14	9	3	2	4	1	7

$A[\text{parent}[i]] \geq A[i]$

- We want to add 15
- We put 15 at the end of the array $A[A.\text{heap-size} + 1]$
- We run max-heapify of $\text{Parent}(A.\text{heap-size} + 1)$

Binary max-heap: max-heap-insert



Binary max-heap: max-heap-insert

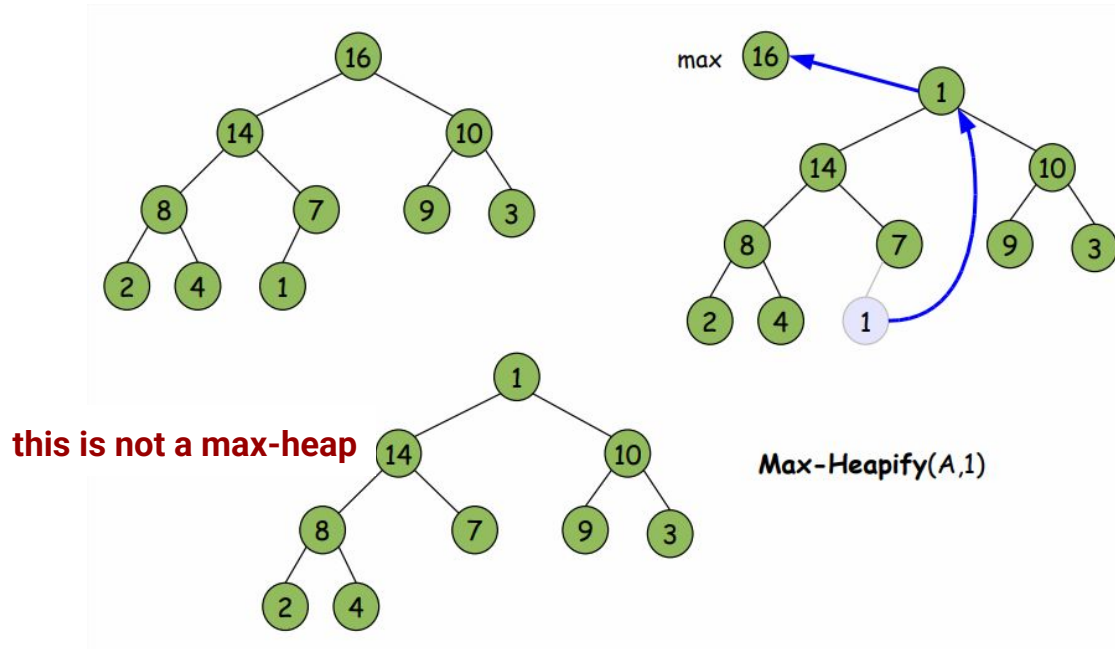
Max-Heap-Insert(A, x)

Complexity

```
{  
    A.heap-size = A.heap-size + 1  
    i = A.heap-size  
  
    A[i] = x  
  
    while (i > 1 or A[Parent(i)] < A[i])  
    {  
        swap(A[i], A[Parent(i)])  
        i = Parent(i)  
    }  
}
```

$O(\log_2 n)$

Binary max-heap: max-heap-extraction



Binary max-heap: max-heap-extract

Max-Heap-Extract(A)

Complexity

```
{  
    if A.heap-size < 1 return error;  
  
    max = A[1]  
    A[1] = A[A.heap-size]  
    A.heap-size = A.heap-size - 1  
    Max-Heap-Heapify(A, 1)  
    return max  
}
```

$O(\log_2 n)$